Full-Stack Apps with Next.js

Pages, Routing, SSR, SSG, Server Actions, Caching, Tailwind, Auth, Deployment

Svetlin Nakov, PhD

Co-founder @ SoftUni

SoftUni AI https://ai.softuni.bg

Agenda

SoftUni AI

1. **Intro to Next.js**: full-stack React framework, App Router, SSR / SSG / CSR, Turbopack
2. **Next.js Project Setup**: npx create-next-app@latest, file structure, App Router folders
3. **Pages, Routing and Layouts**: file-system routing, dynamic routes, API routes, nested layouts
4. **Client Components vs. Server Components**: "use client", hydration, SEO, data access
5. **Rendering Modes**: SSR (dynamic), SSG (static), ISR (hybrid), caching and revalidation
6. **Server Actions**: "use server", form submissions, DB mutations without REST endpoints
7. **Styling with Tailwind**: utility classes, responsive design, component styling
8. **Auth, JWT, Sessions and Middleware**: middleware.ts, roles, protected routes
9. **Performance Optimizations**: <Link> prefetch, <Image>, WebP, caching strategies
10. **Suspense and Streaming**: loading.tsx, <Suspense>, progressive HTML delivery
11. **Full-Stack App**: Neon DB + Drizzle ORM + Next.js + Tailwind, deploy to Netlify

Sli.do Code

AppsAI

Join at

slido.com

#AppsAI

Breaks

20:00 / 21:00

Introduction to Next.js

Full-Stack React Framework for Modern Web Apps, with Server-Side Rendering

What is Next.js?

SoftUni AI

- **Next.js** is a **full-stack framework** for modern Web apps
  - Built on top of **React**, by **Vercel** – https://nextjs.org
  - Adds **routing**, **server rendering**, **data fetching**, **caching**

- Why Next.js apps are **fast**?
  - **Server rendering** → server sends HTML → faster load + SEO
- **Front-end** + **back-end** into a single app (React + TS)
  - **Front-end**: React components + UI rendering
  - **Back-end**: server actions + API routes
  - Direct **database access** from server components

Why Next.js for React Devs?

SoftUni AI

⚛ Plain React Apps

Client-side rendering

- **SEO-unfriendly** – search engines initially see empty HTML
- **Slower first paint** – JavaScript must download and execute first
- **Manual setup** – routing, data fetching, builds require configs
- **No server-side capabilities** – no built-in backend or SSR support
  - Apps need separate back-end

▲ Next.js Apps

Server rendering + full-stack

- **SEO-friendly** – content comes as HTML from the server
- **Faster initial load** – pre-rendered pages improve performance
- **Built-in routing** – no manual setup
- **Built-in optimizations** – image handling, caching, and code splitting
- **Serverless deployment** – back-end + front-end both run in Vercel / Netlify

Rendering Modes in Next.js

SoftUni AI

🌐 CSR (Client-Side Rendering)

- Browser receives **empty HTML + JS**, then **React builds the UI**
- **Not SEO-friendly**, slower first paint

⚙️ SSR (Server-Side Rendering)

- Server builds **HTML on each request**, sent ready to browser
- **SEO-friendly**, always fresh data

📄 SSG (Static Site Generation)

- HTML built **once at build time**, served from CDN
- **Fastest**, SEO-friendly, ideal for marketing pages

🔄 ISR (Hybrid Rendering)

- **Incremental Static Regeneration** — best of SSG + SSR
- **Static**, but auto-**revalidates** in the background (e.g. in 5 mins)

Next.js Key Components

SoftUni AI

- **App Router** – file-system based routing in the **app/** folder
- **React** – latest React with **server components** support
- **Turbopack** – next-gen Rust-based bundler, fast and stable
- **Server Components** – render on the server, can access DB directly
- **Client Components** – render in the browser, handle interactivity
- **API Routes** – backend endpoints inside Next.js
- **Server Actions** – call server functions directly from forms
- **Middleware** – runs before each request (auth, redirects, logging)
- **Built-in optimizations** – **<Image>**, **<Link>**, fonts, caching
- **Streaming & Suspense** – progressive UI loading (with indicator)

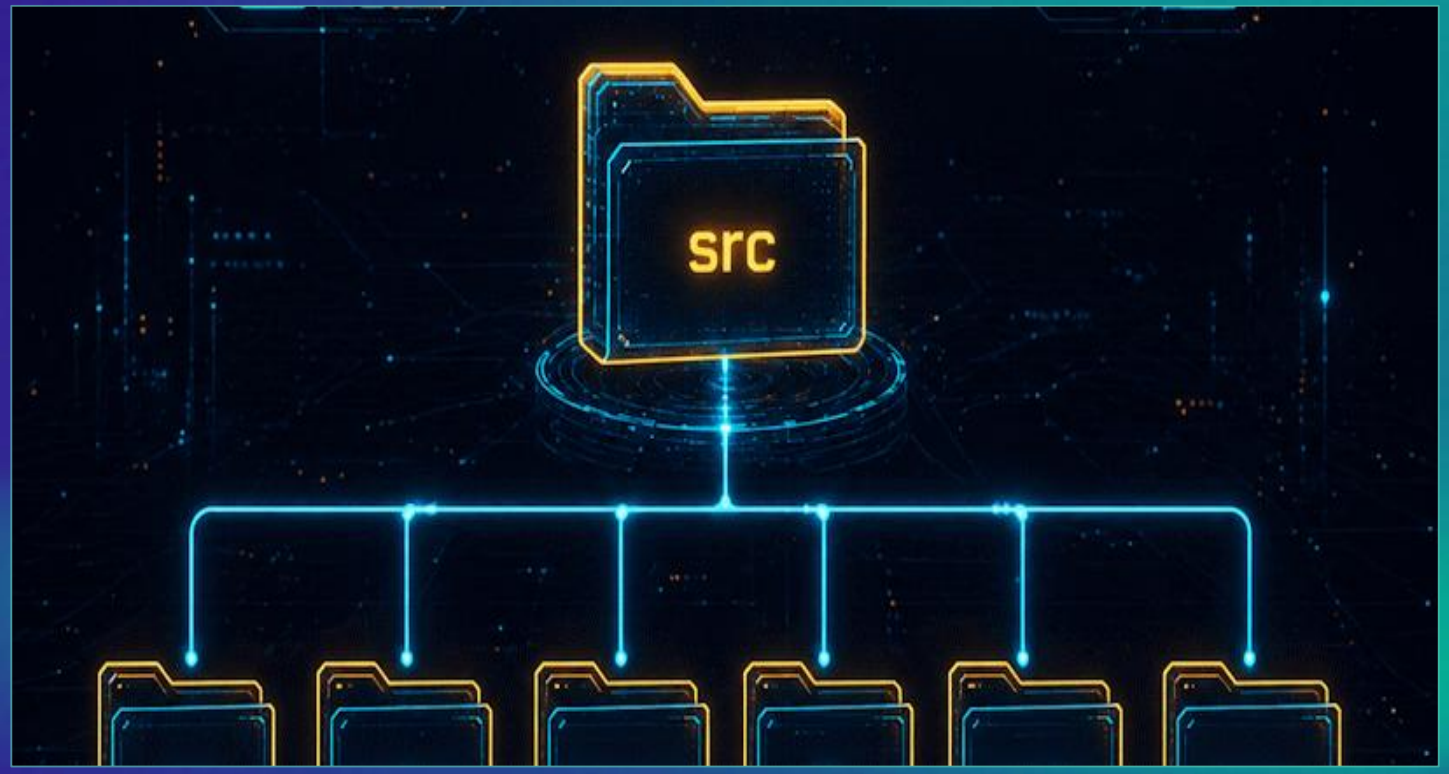
Next.js Versions

SoftUni AI

- Next.js has two major variants:
  - **Pages Router** (Next.js < v13) | **App Router** (Next.js v13+)
- **Old Next.js**: pages router with client-first rendering in **/pages**, no server actions → legacy, don't use for new projects
- **New Next.js**: app router with server-first rendering in **/app**, nested layouts, server actions, API routes, loading / error UI
  - Use the **modern Next.js**, with server-side rendering!
- Note: **AI dev agents** can **confuse** Next.js versions → incompatible code / config / libraries, hallucinations

Next.js Project Setup

Scaffolding, File Structure, App Router



Creating a Next.js Project

SoftUni AI

🚀 Scaffold a **Next.js** project with the official CLI:

```
npx create-next-app@latest .
```

▶️ Then **run the dev server** → http://localhost:3000

```
npm run dev
```

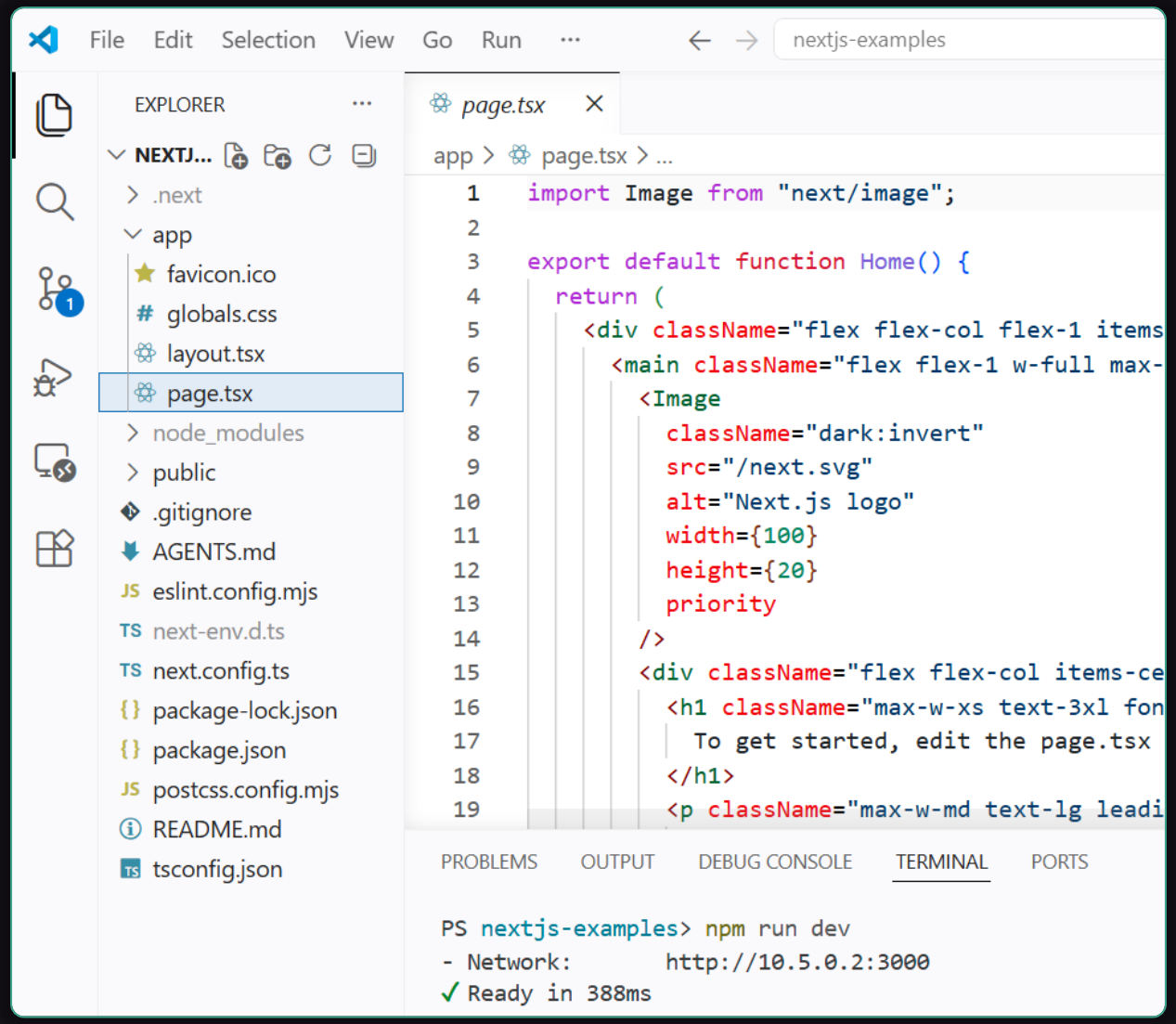
⚙️ Recommended setup:

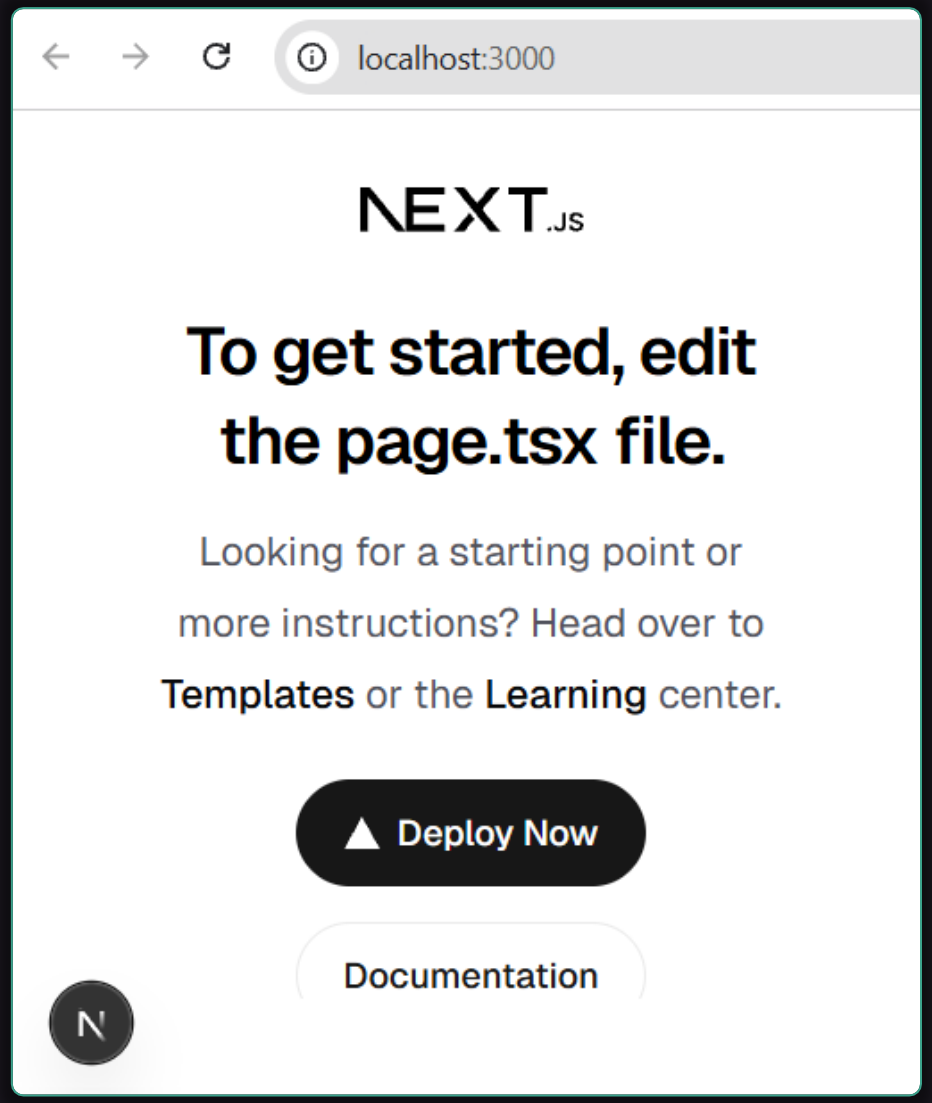
- **TypeScript**: Yes ✓
- **ESLint**: Yes ✓
- **Tailwind**: Yes ✓
- **App Router**: Yes ✓
- **Turbopack**: Yes ✓

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

PS nextjs-examples> npm run dev
> next dev

▲ Next.js 16.2.4 (Turbopack)
- Local:         http://localhost:3000
- Network:       http://10.5.0.2:3000
✓ Ready in 388ms
```

Running a Next.js Project

SoftUni AI





Next.js – Project Structure

SoftUni AI

- ⚙️ `package.json` – Node.js config
- 📁 `app/` – routes, layouts, pages
- 📄 `layout.tsx` – root layout
- 📄 `page.tsx` – home page "/"
- 🖼️ `public/` – static assets
- 🧩 `components/` – reusable React components
- 🛠️ `lib/` – helpers, utils, db client
- 🔐 `.env` – secrets (DB URL, JWT)
- ⚙️ `next.config.ts` – Next.js config
- ⚙️ `tsconfig.json` – TypeScript config

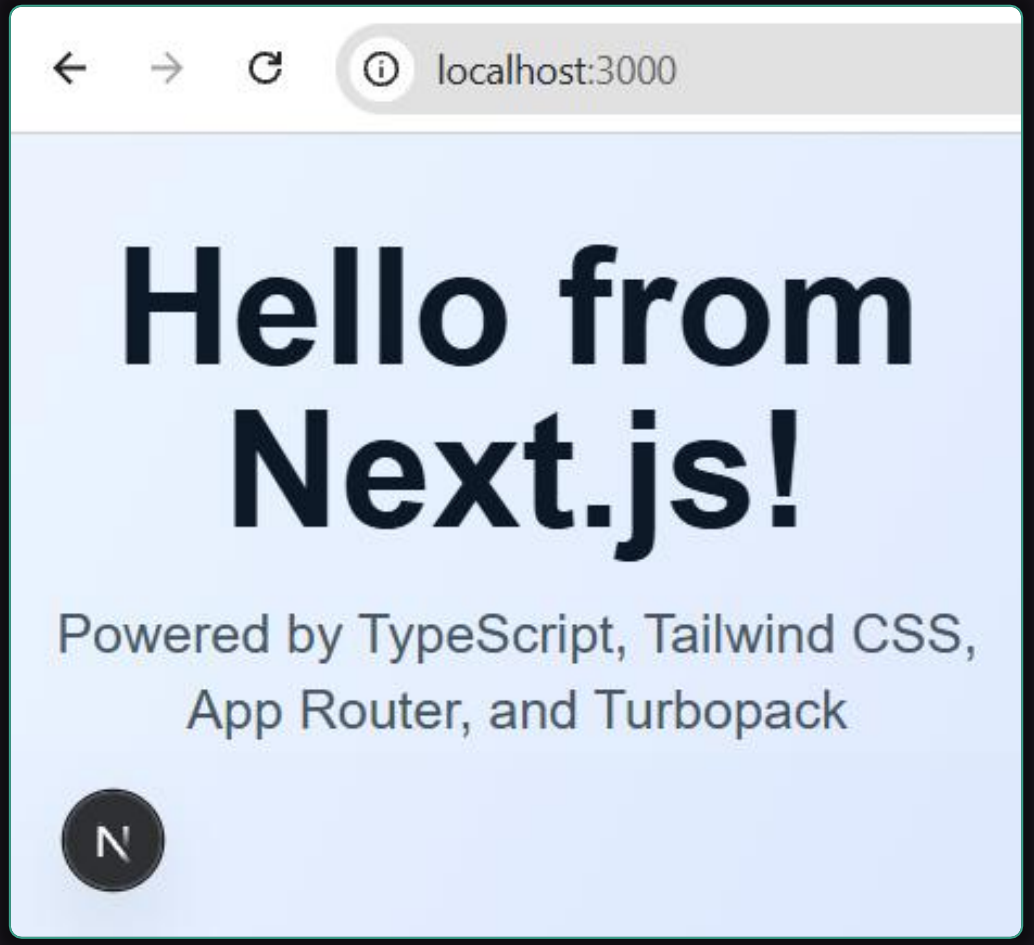
```
my-nextjs-app/
├── package.json
├── app/
│   ├── layout.tsx
│   ├── page.tsx
│   └── globals.css
├── public/
├── components/
├── lib/
├── .env
├── next.config.ts
├── tsconfig.json
└── AGENTS.md / CLAUDE.md
```

Live Demo – First Next.js App

SoftUni AI

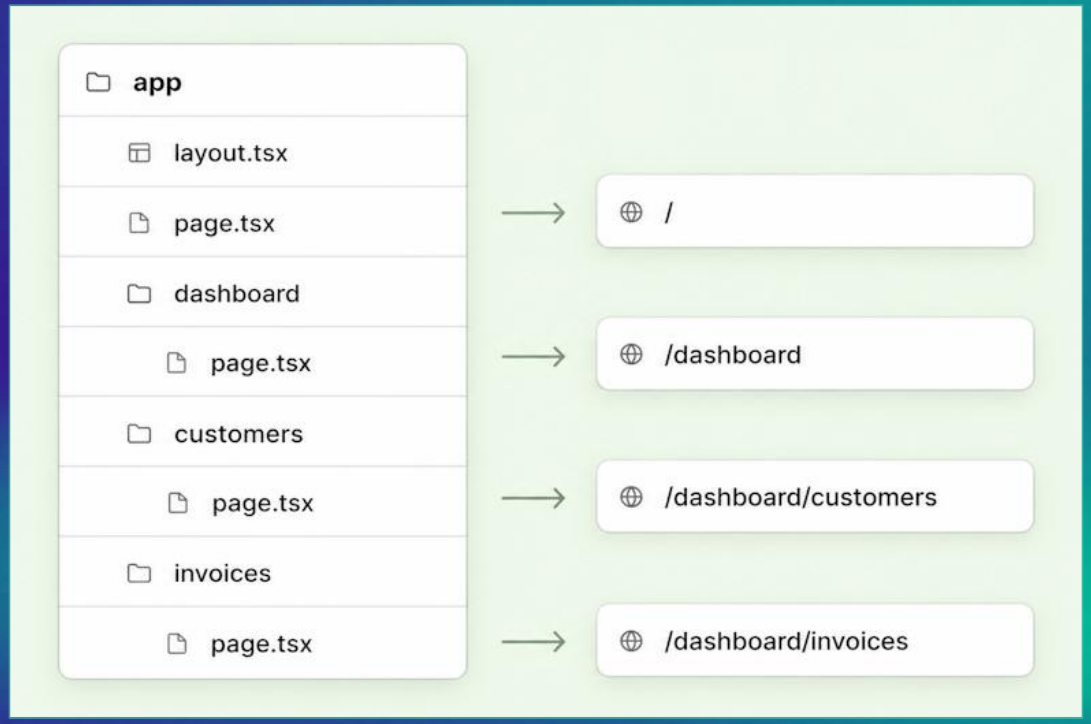
🤖 **AI prompt** to scaffold and run a starter Next.js app:

Scaffold a **new Next.js app**:
- Use **TypeScript**, **Tailwind**, **App Router**, **Turbopack**
- Edit `app/page.tsx` to show: "*Hello from Next.js!*"
- Start the **dev server** and open the app in the browser



Pages, Routing and Layouts

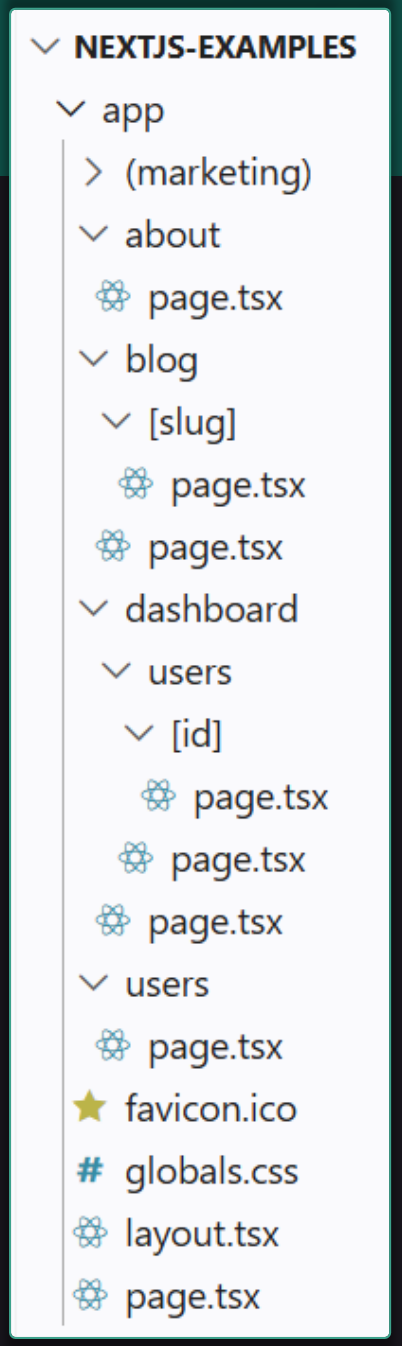
App Router, File-System Routing, Dynamic Routes, API Routes



File-System Routing

- In Next.js each **folder** in `app/` = a route segment
  - `page.tsx` = the UI shown at that URL
  - Brackets `[id]` = **dynamic segment**
- **No router config** — the file tree is the router

```
app/page.tsx                          →  /
app/about/page.tsx                    →  /about
app/blog/page.tsx                     →  /blog
app/blog/[slug]/page.tsx              →  /blog/hello
app/dashboard/page.tsx                →  /dashboard
app/dashboard/users/page.tsx          →  /dashboard/users
app/users/[id]/page.tsx               →  /users/42
app/(marketing)/page.tsx              →  (group, no URL)
```



SoftUni AI

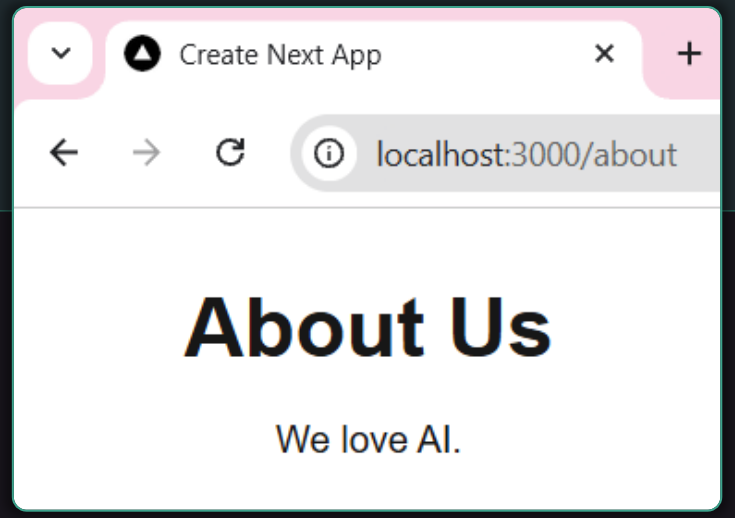
Creating a Simple Page

SoftUni AI

🤖 **AI prompt** to create a new app page:

Create a **new page**: **/about**
- Heading **<h1>**: About Us
- A paragraph: about text
- Style with **Tailwind**: centered, padded

```
export default function AboutPage() {
  return (
    <main className="p-8 text-center">
      <h1 className="text-4xl font-bold">
        About Us
      </h1>
      <p className="mt-4">We love AI.</p>
    </main>
  );
}
```



Dynamic Routes

SoftUni AI

🛣️ Folders wrapped in `[brackets]` become **route parameters**

- 📂 `app/news/[id]/page.tsx` matches **/news/1**, **/news/42**, …
- ⚡ `params` is **async**

🌐 Use `[...slug]` for **catch-all routes**

- **app/docs/[...slug]/page.tsx** matches **/docs/react/hooks**, **/docs/react/hooks/use-effect**

```
type Props = {
  params: Promise<{ id: string }>;
};
```

```
export default async function
NewsItem({ params }: Props) {
  const { id } = await params;
  return <h1>Article #{id}</h1>;
}
```

Live Demo – Page with Route

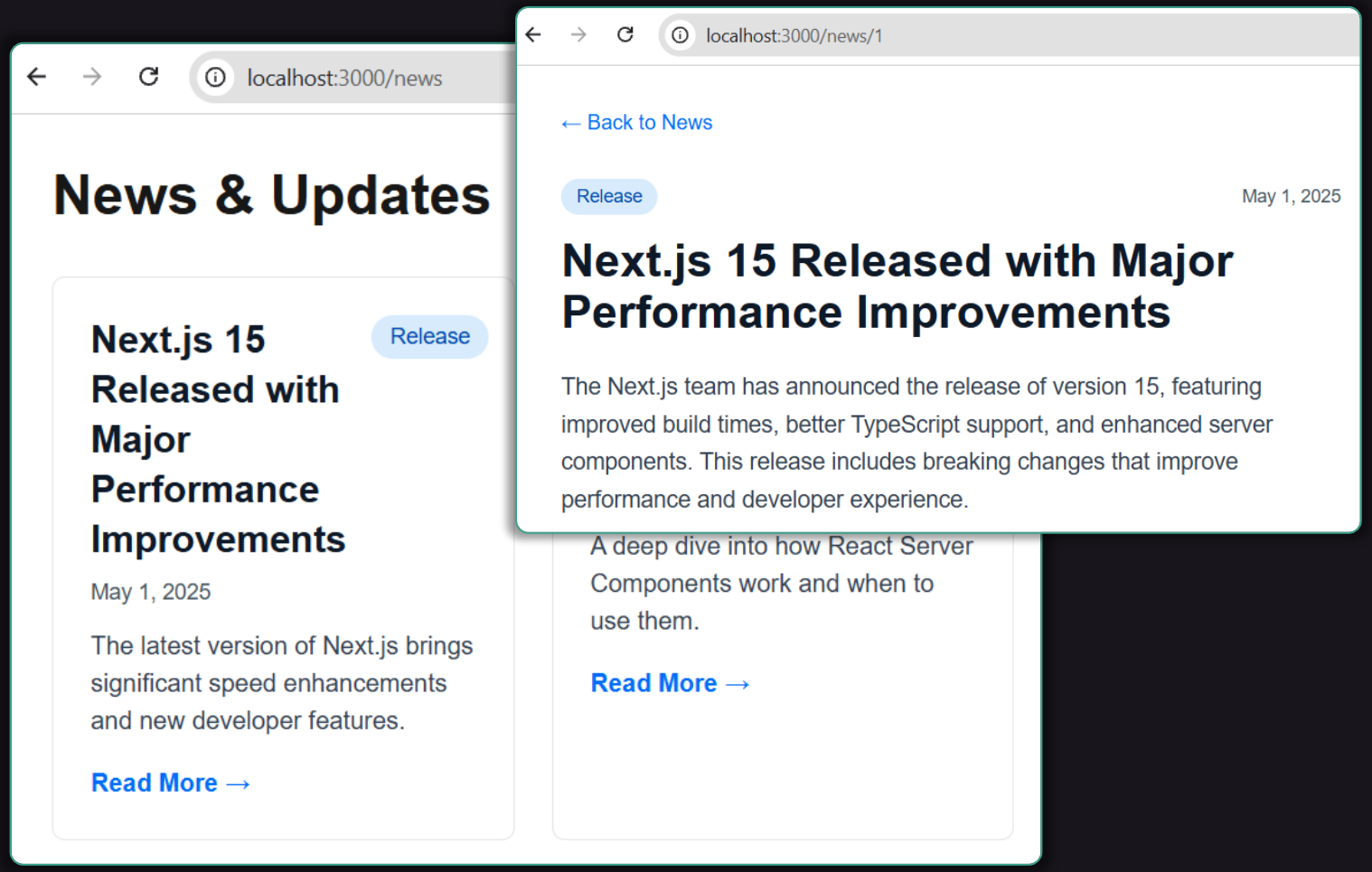
SoftUni AI

🤖 **AI prompt** to create a page with dynamic routing:

Create a page **/news** → render a list of **news**

Create a dynamic route: **/news/[id]** → render **news item**

Define sample list of news in `**news-data.ts**`.



API Routes

SoftUni AI

- Server-only endpoints in `app/api/…/route.ts`
- File exports HTTP method functions: `GET`, `POST`, `PUT`, `DELETE`
- Call from the client with `fetch("/api/hello")`

```
export async function GET() {
  return NextResponse.json({
    message: "Hello API!"
  });
}
```

```
export async function POST(req: Request) {
  const body = await req.json();
  return NextResponse.json({body });
}
```

```
async function loadApiHello() {
  let data = await (await fetch("/api/hello")).json();
  setMessage(data.message);
}
```

Live Demo – API Routes

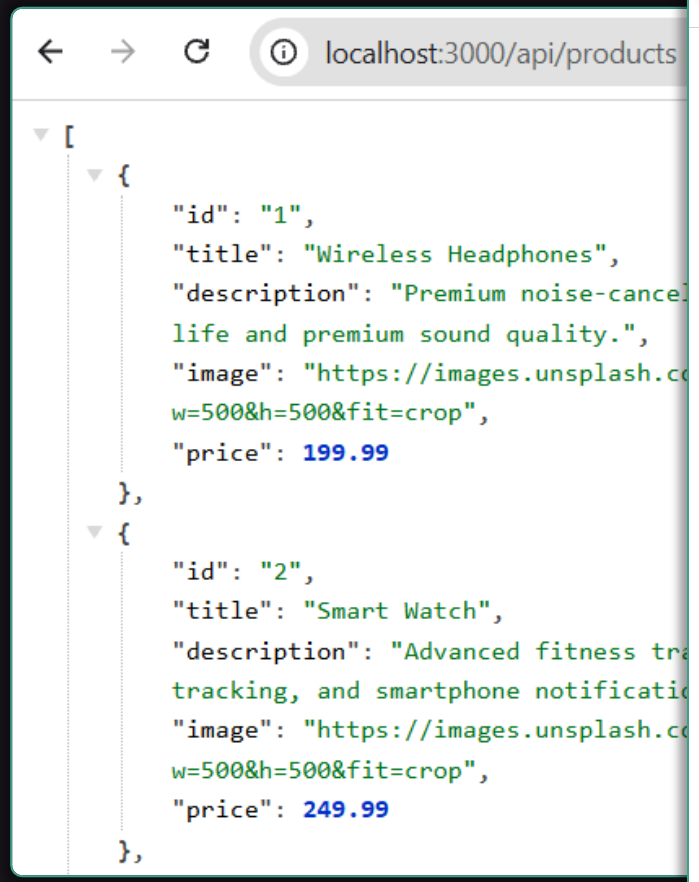
SoftUni AI

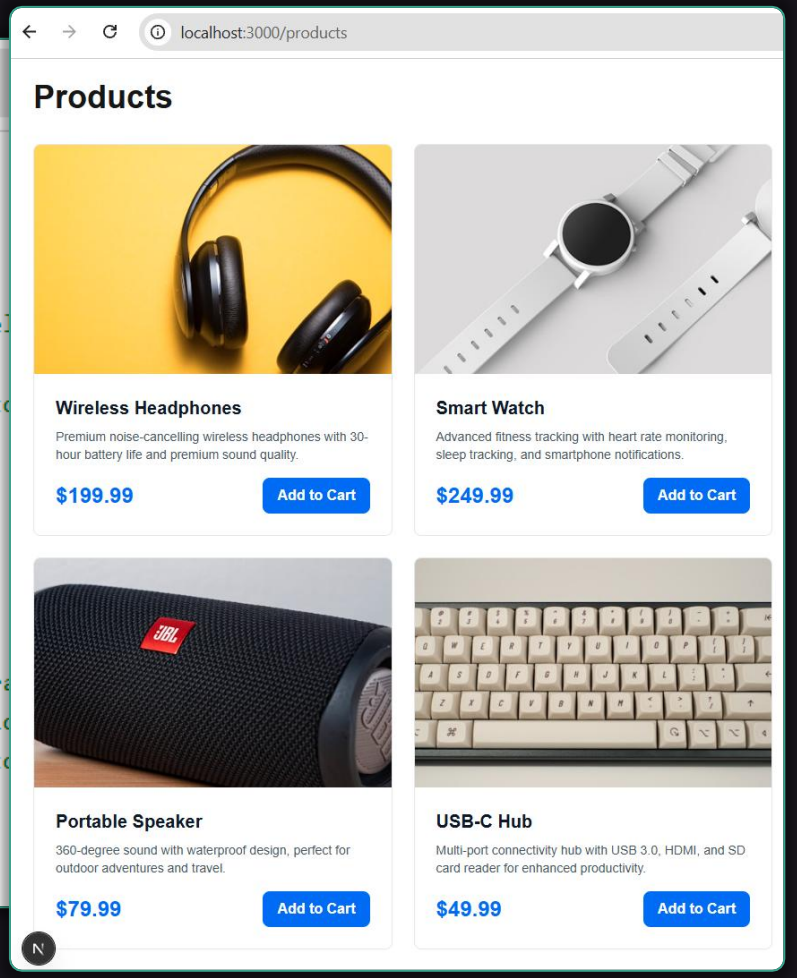
🤖 AI prompt to create API routes + client page:

Create an API route **GET /api/products** → return a list of products

Define sample products in `**products-data.ts**` (title, description, image)

Create a page **/products** → display the products from the API as cards





Layouts

SoftUni AI

- `layout.tsx` = shared UI wrapper for a route + children
  - **Root layout** `app/layout.tsx` wraps the entire app
  - **Nested layouts**: `app/dashboard/layout.tsx` wraps `/dashboard/*`
- **Layouts persist** on navigation — no re-render of the wrapper

```
export default function
RootLayout({ children }: {
  children: React.ReactNode;
}) {
  return (
    <html lang="en">
      <body>
        <Header />
        {children}
        <Footer />
      </body>
    </html>
  );
}
```

Live Demo – Site Layout

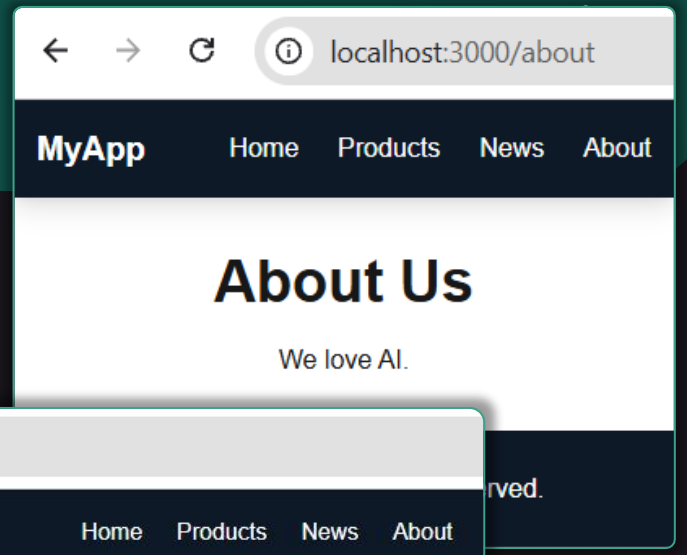
🤖 **AI prompt** to create site layouts:

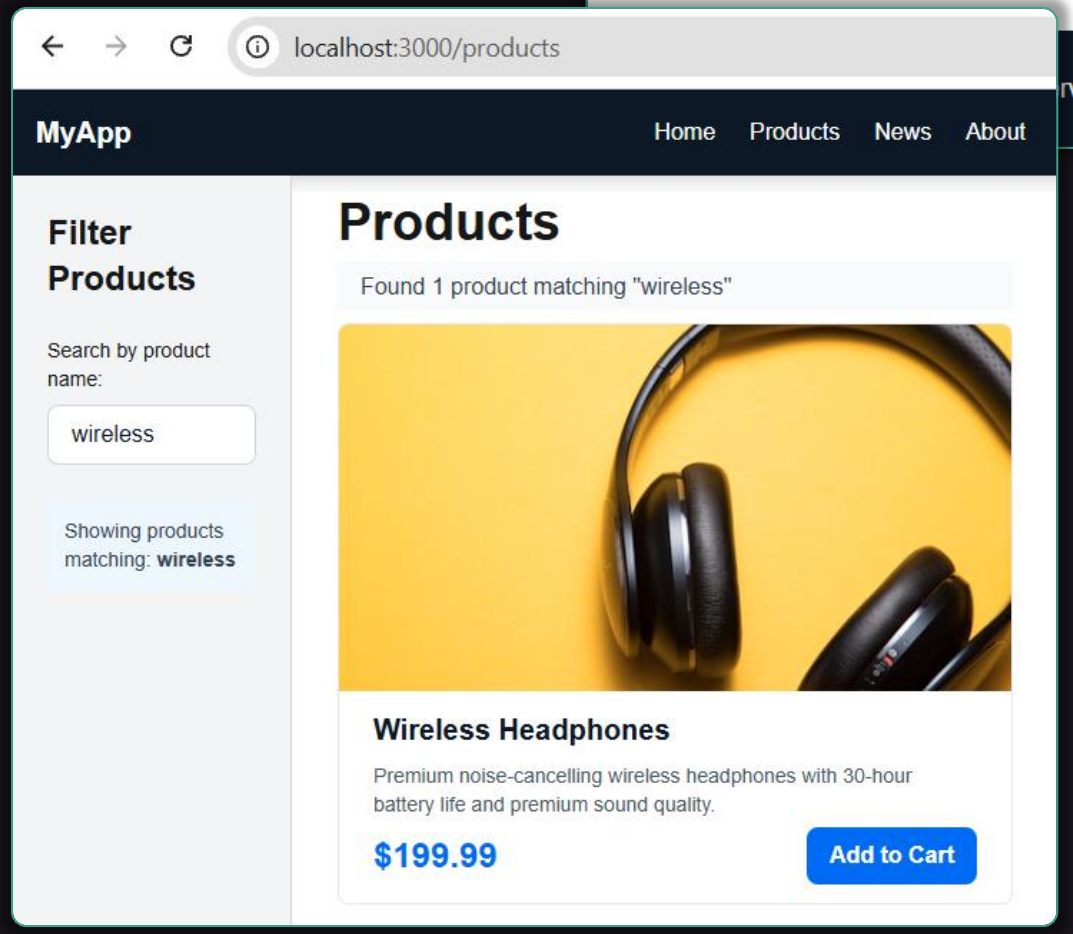
Implement a **site layout**:

- Top **nav bar with links**: Home, Products, News, About
- **Footer** with copyright notice
- Apply to all pages via `app/layout.tsx`

Create **nested layout** at `app/products/layout.tsx`

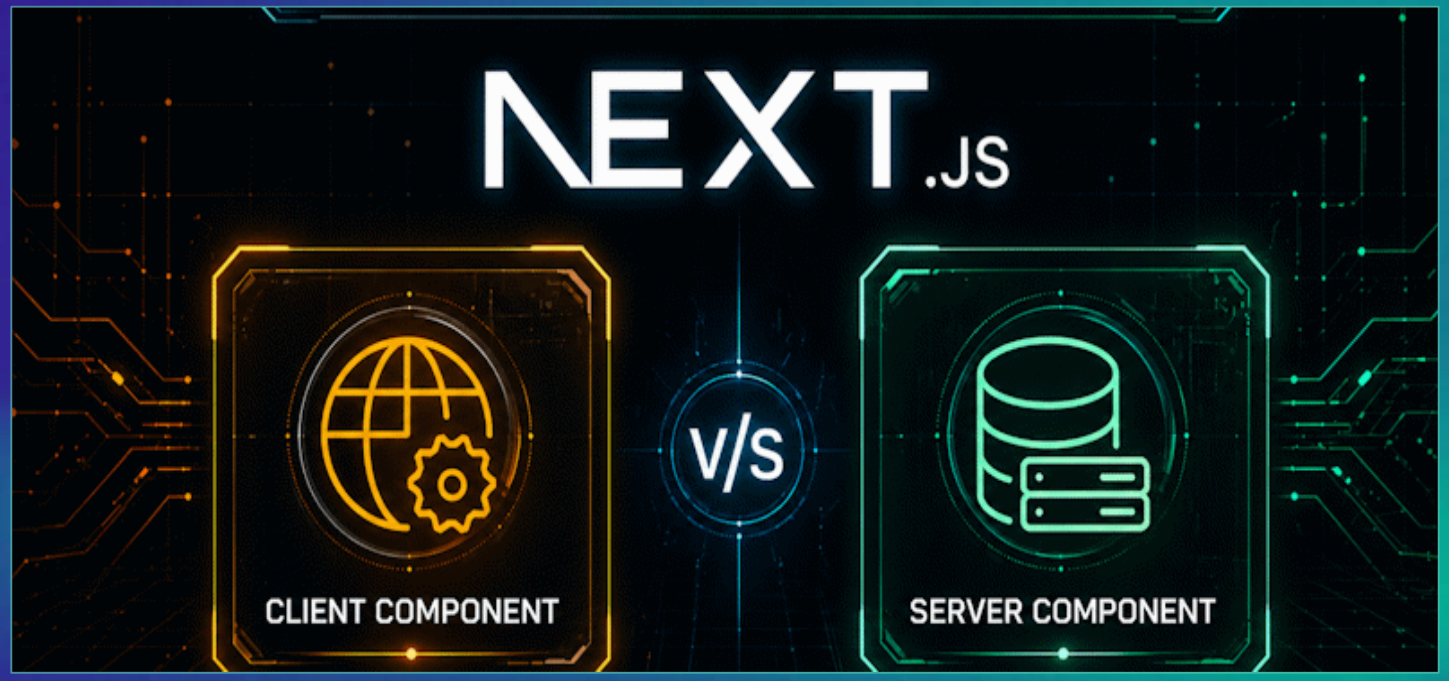
- Display a sidebar: filter products by substring





Client Components vs. Server Components

Rendering Modes, Hydration, SEO, Data Access



Server and Client Components

SoftUni AI

- **Client components**
  - Run in the **browser**
  - Server sends **HTML+JS code**
  - **React** renders / updates the UI in the Web browser
- **Server components** (default in Next.js)
  - Run on the **server**
  - Server sends **HTML code** + component tree (no JS)
  - **Browser** renders the HTML

```
"use client"
export function Button() {
  return <button onClick={() =>
   alert("Hi")}>Click</button>
}
```

```
export default async function Page() {
  const posts = await getPosts();
  return <div>{posts.count} posts</div>
}
```

```
export async function getPost() {
  return db.posts.select();
}
```

Server Components by Default

SoftUni AI

📌 **By default**, components in Next.js are "**server components**"

- Unless declared **"use client"**, React components run server-side

🖥️ **Server components**:

- Render on the server → **HTML sent to the browser**
- **SEO-friendly** – content visible in **[View Page Source]** / **[Ctrl+U]**
- Can access the **database** and **app secrets** directly
- **Automatically cached** by default (unless "no-cache")
- Cannot use `useState()`, `useEffect()`, or browser APIs
- Cannot handle browser events like `` `onClick` ``

Server Component – Example

SoftUni AI

- **Server components** run on the server
  - **Query DB directly** → return HTML
  - No `useEffect()`, no `fetch('/api/…')` needed
  - The component is `async` → await DB queries inline

```
// app/books/page.tsx (Server Component)
import { db } from "@/lib/db";
```

```
export default async function BooksPage() {
  const books = await db.select()
    .from(booksTable);
  return (
    <ul>
      {books.map((b) => (
        <li key={b.id}>{b.title}</li>
      ))}
    </ul>
  );
}
```

Client Components

SoftUni AI

- **Client components**
  - Declared with `"use client"` at the top
  - Ship as HTML + JS to the browser → **hydrated** by React to add interactivity
  - Used for **client-side interactions**
  - Can use `useState()`, `useEffect()`, `onClick`, browser APIs
  - **NOT SEO-friendly** — interactive parts render in the browser
  - Use for: **forms**, **buttons**, **modals**, **charts** — anything interactive
- **Rule**: keep the client bundle small
  - Use client components **only where needed** (client-side interactions)

```
"use client";
export default function ClickMe() {
  return (
    <p onClick={() =>
      console.log("Msg in the
      browser")}>Click me</p>
  );
}
```

Client Component – Example

SoftUni AI

- Marked with `"use client"` → runs in the **browser**
- A server page can still **import and render** `<Counter />`

```
// app/page.tsx
<h1>Home Page</h1>
<Counter />
```

- Server sends initial HTML + JS bundle → React **hydrates** in browser

```
// app/components/Counter.tsx
"use client";
import { useState } from "react";

export default function Counter() {
  const [count, setCount] =
    useState(0);
  return (
    <button onClick={() =>
        setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}
```

Server vs. Client – Side by Side

SoftUni AI

🖥️ Server Component

used by default in Next.js

- Rendered on the **server**
- `async function` allowed at top level
- No hooks, no events
- **Direct DB** / **API** / **files** access
- **SEO-friendly** HTML
- Better **performance** (load faster)

💻 Client Component

"use client"

- Pre-rendered at the **server**
  - **Hydrated** in the browser
- No `async` at top level
- Uses hooks: `useState()`, `useEffect()`
- Handles browser events: `onClick`, `onChange`

Mix freely: server components can import client components (not vice-versa)

Hydration Explained

SoftUni AI

- **Hydration** = making static HTML interactive
- **Client components flow**:
  1. 🖥️ **Server renders HTML** → sent to browser
  2. ⚡ Browser displays static HTML — **fast first paint**
  3. 📦 **JS bundle downloads** (component JS code)
  4. ⚛️ React **hydrates** → attaches event listeners + restores state
  5. ✨ Page becomes **interactive**
- Only **client components** need hydration
- **Server components = pure HTML**, zero JS shipped for them

Mixing Server and Client Components

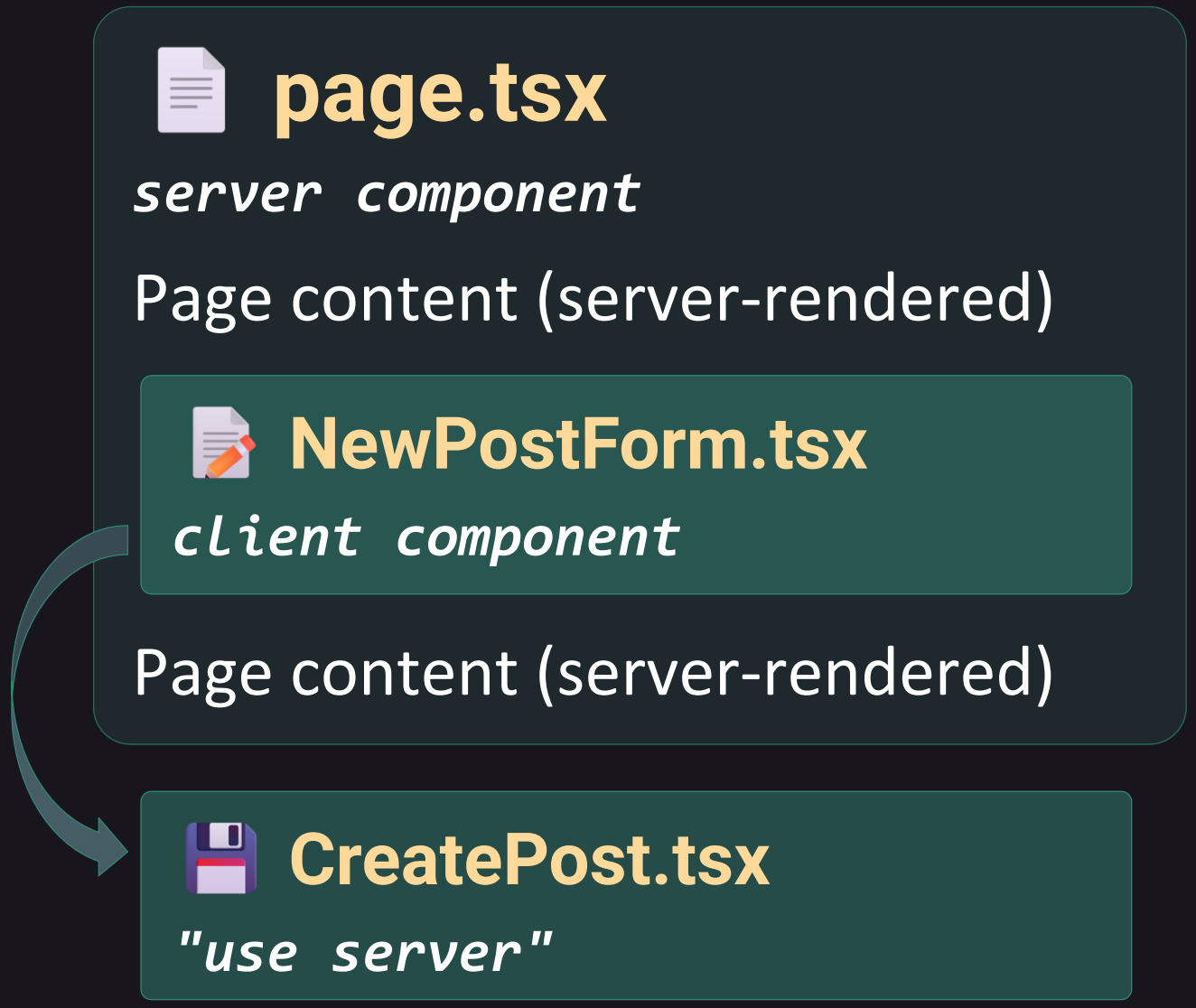
SoftUni AI

- When to use **server components**?
  - Page main content
  - Texts, visible for search engines: blog posts, news, product descriptions
  - Layouts, headers, footers, sidebars
  - All other page content
- When to use **client components**?
  - Only when server-rendering does not work
  - Interactive elements: client-side filters, modal popups, tabs, like box, …
  - Forms with live validation
  - Everything, which needs in-browser event handlers

Mix SSR with Client Components

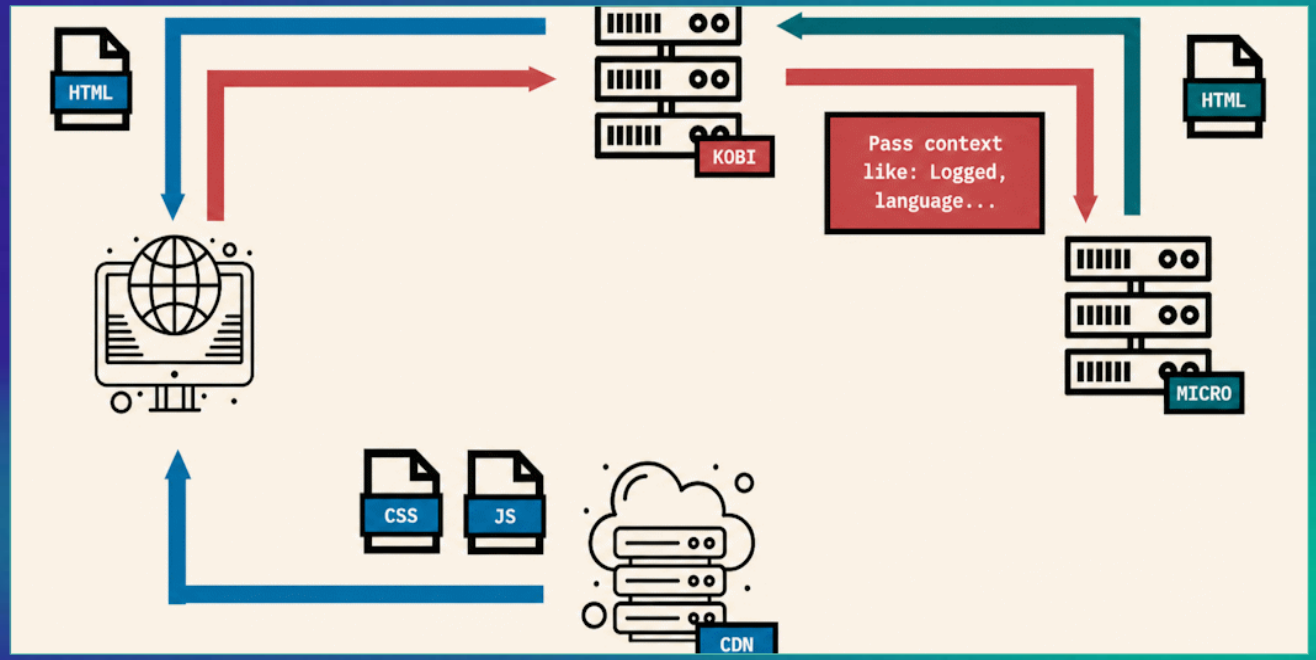
SoftUni AI

- How to implement a "Create a Post" page?
  - **page.tsx** → server component
  - **NewPostForm.tsx** → client component (interactive form)
  - **createPost.tsx** action → server-side mutation



Static, Dynamic and Hybrid Rendering

SSG, SSR, ISR, Caching and Revalidation



Three Server Rendering Strategies

SoftUni AI

📄 SSG (Static Site Generation)

- HTML generated at **build time**, served from CDN
- **Fastest**, **cheapest**
- Perfect for **About**, **Docs**, **Landing Pages**

⚙️ SSR (Server-Side Rendering)

- HTML generated on **every request**
- Use for **user-specific** or **rapidly changing** data
- **Always fresh**, SEO-friendly

🔄 ISR (Incremental Static Regeneration)

- **Hybrid approach**: SSG + SSR combined
- Static, but **auto-revalidates** every X seconds
- Best of both: **fast AND fresh**

How Next.js Picks the Rendering Mode?

- Next.js decides **automatically** based on what the route uses:
  - Uses `cookies()`, `headers()`, or dynamic params → **SSR**
  - Uses `fetch(url, { next: { revalidate: 60 } })` → **ISR**
  - Everything static and cacheable → **SSG**
- Next.js rendering rules:
  - **Anything dynamic** in the route → whole route becomes dynamic (SSR)
  - **Everything cacheable** → route stays static (SSG)
  - Used `` `revalidate` `` → hybrid (ISR)

View Rendering Mode for Routes

SoftUni AI

- See the **rendering mode** for each route in Next.js:

```
npm run build
```

```
PS nextjs-examples> npm run build

> nextjs-examples@0.1.0 build
> next build

▲ Next.js 16.2.4 (Turbopack)

  Creating an optimized production build ...
✓ Compiled successfully in 3.3s
✓ Finished TypeScript in 2.6s
✓ Collecting page data using 7 workers in 899ms
✓ Generating static pages using 7 workers (9/9) in 292ms
✓ Finalizing page optimization in 22ms

Route (app)                 Revalidate  Expire
┌ ○ /
├ ○ /_not-found
├ ○ /about
├ ƒ /api/products
├ ○ /client-comp
├ ○ /news                          5m      1y
├ ƒ /news/[id]
└ ○ /products

○  (Static)   prerendered as static content
ƒ  (Dynamic)  server-rendered on demand
```


```
Route (app)                 Revalidate  Expire
┌ ○ /
├ ○ /_not-found
├ ○ /about
├ ƒ /api/products
├ ○ /client-comp
├ ○ /news                          5m      1y
├ ƒ /news/[id]
└ ○ /products

○  (Static)   prerendered as static content
ƒ  (Dynamic)  server-rendered on demand
```

SSG – Static Page Example

SoftUni AI

- **Static pages** (SSG)
  - **Built once**, served forever (until next deploy)
  - Cached by the CDN
  - Perfect for **rarely-changing** content
- Run `npm run build` → see `/about (Static)`
- **Default mode** if no dynamic APIs are used

```
// app/about/page.tsx → SSG by default
export default function AboutPage() {
  return (
    <main>
      <h1>About Us</h1>
      <p>Founded in 2013…</p>
    </main>
  );
}
```

SSR – Dynamic Page Example

SoftUni AI

- **Dynamic pages** (SSR)
  - **Re-rendered** on every request
  - `cache: "no-store"` → always fresh
  - Each visitor sees **live data**
- `force-dynamic` makes a route SSR

```tsx
// app/now/page.tsx
export default async function NowPage() {
  const res = await fetch(
    "https://api.weather.com/Rome/now",
    { cache: "no-store" });
  const data = await res.json();
  return <h1>{data.temp} °C</h1>;
}
```

```tsx
// app/now/page.tsx
export const dynamic = "force-dynamic";
```

ISR – Hybrid Rendering

SoftUni AI

- **Cached pages with revalidate** (ISR)
  - **Static page**, with periodic **refresh**
- First visit after **60s** triggers a rebuild
  - Everyone else gets the **cached version** → very fast
- **Best of both worlds**: fast AND fresh

```
// app/news/page.tsx
export default async function NewsPage() {
  const res = await fetch(
    "https://api.news.com/top",
    { next: { revalidate: 60 } }); // 60s
  const articles = await res.json();
  return (
    <ul>
      {articles.map(a =>
        <li key={a.id}>{a.title}</li>
      )}
    </ul>
  );
}
```

Live Demo – Compare SSG, SSR, ISR

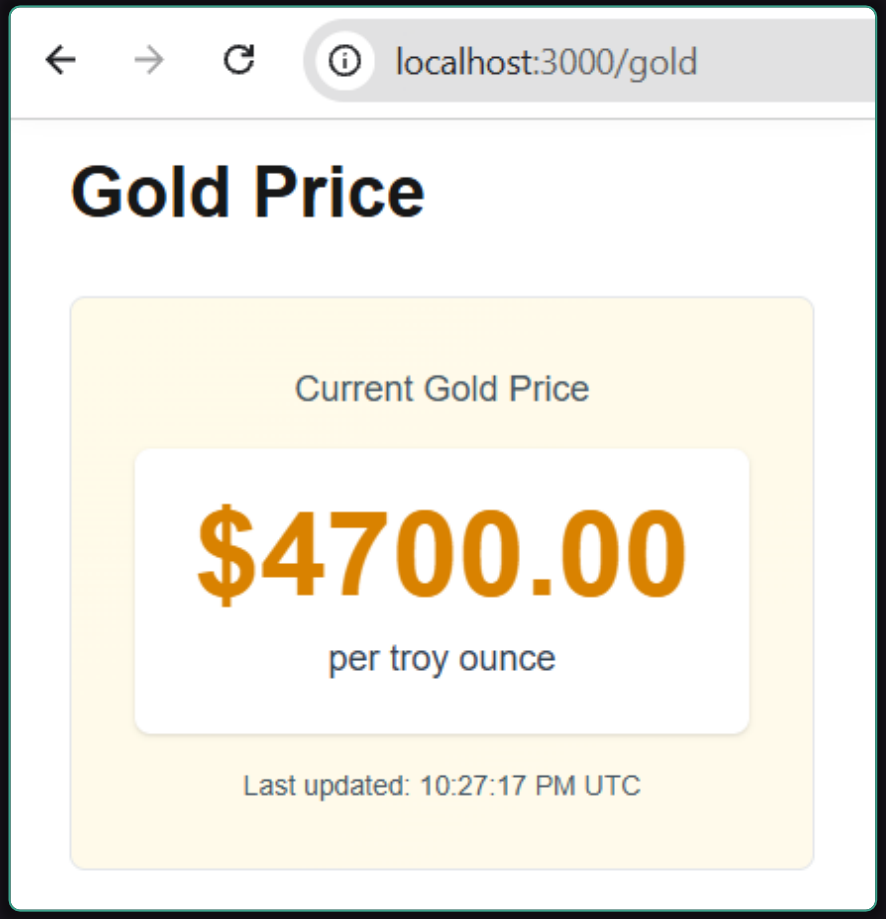
SoftUni AI

🤖 **AI prompt** to create static and dynamic pages:

Create **3 pages** in the same Next.js app:

- **/contacts** -> **static** (SSG), shows "**Contact Us**" (sample content)
- **/time** -> **dynamic** (SSR), shows the current **server time** on every refresh
- **/gold** -> **cached** (ISR), fetches **gold price** from https://api.gold-api.com/price/XAU, revalidates every 30s

🔍 Run `npm run build` to see **static / dynamic** routes



Server Actions in Next.js

Form Submissions without REST Endpoints

What are Server Actions?

SoftUni AI

- ⚡ **Server actions** in Next.js
  - ⚡ Functions marked `"use server"` that run on the **server**
  - 📝 Called directly from **forms** or **client components**
  - 🚫 **No `/api` routes needed** for simple mutations
- 🛠️ **Server actions** can:
  - Access the **database** directly
  - Read **secrets and `.env` variables**
  - **Revalidate caches** after a mutation
  - **Redirect** after submission

```
"use server"
export async function
submitForm(form: FormData) {
  // db.insert (form data)
}
```

Server Action – Example

SoftUni AI

- **Action file**: declare with `"use server"`
- **Page**: pass it to `<form action={…}>`

```
// app/dashboard/actions.ts
"use server";
import { db } from "@/lib/db";

export async function addBookmark(
    form: FormData) {
  const url =
    form.get("url") as string;
  await db.insert(bookmarksTable)
    .values({ url });
}
```

```
// app/dashboard/page.tsx
import { addBookmark } from
"./actions";

export default function
Dashboard() {
  return (
    <form action={addBookmark}>
      <input name="url" />
      <button type="submit">
       Add</button>
    </form>
  );
}
```

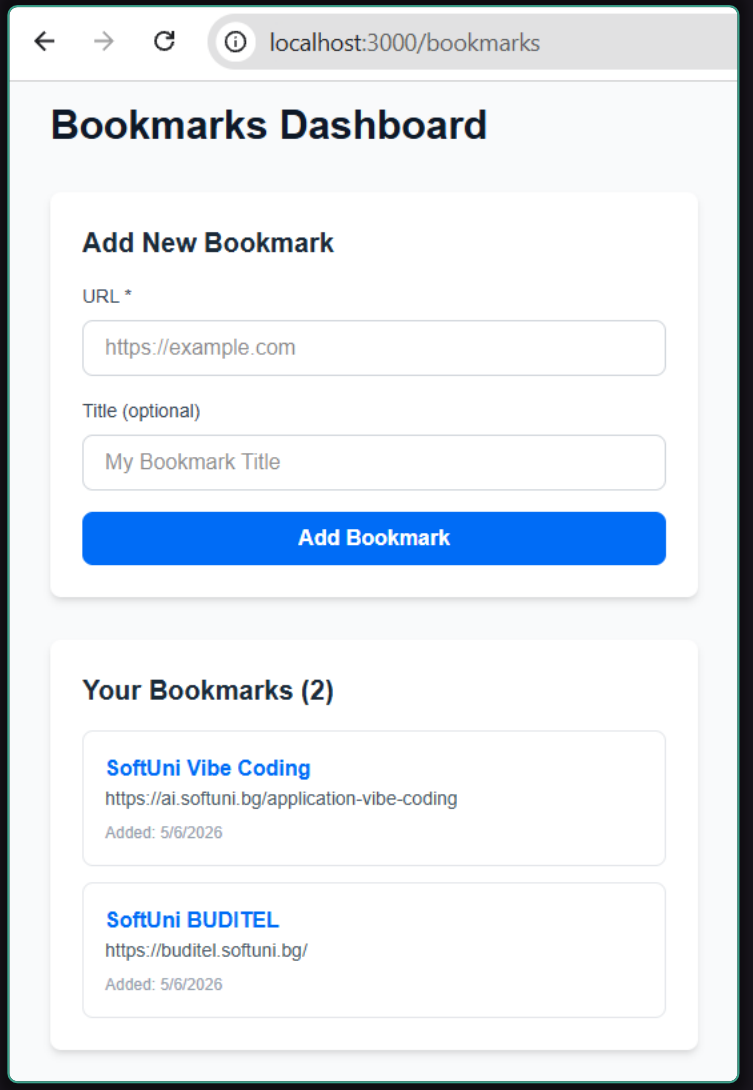
Live Demo – AddBookmark Form

SoftUni AI

🤖 **AI prompt** to build a dashboard with **server actions**:

Build a **bookmarks dashboard** in `app/bookmarks/page.tsx`:

- For simplicity, keep all bookmarks in a local storage file: `**app-data.json**`
- **Server component** lists all bookmarks from the storage
- `<form>` submits to **server action** `addBookmark`
- The action inserts the **URL into bookmarks**
- Call `revalidatePath("/dashboard")`
- No `useState()`, `useEffect()`, or `/api` route



Styling Next.js Apps with Tailwind

Utility-First CSS, Responsive Design, Component Styling

Tailwind CSS in Next.js

SoftUni AI

- Tailwind = **utility-first CSS framework**
- **Pre-configured** by **`create-next-app`**
- Write styles as classes, **directly in JSX**
- **No separate CSS**, no naming conventions
- **Responsive** with `sm:`, `md:`, `lg:` prefixes

- **Tailwind** is quite suitable for **AI-first development**
  - Less files to manage → less complexity for the AI dev agents

```
<button className="
  bg-blue-600 hover:bg-blue-700
  text-white font-semibold
  px-4 py-2 rounded-lg
  transition-colors">
  Sign Up
</button>
```

Common Tailwind Utilities

SoftUni AI

📐 Layout

```
flex
grid
gap-4
items-center
```

📏 Spacing

```
p-4
px-6
my-2
space-y-4
```

🔤 Typography

```
text-lg
font-bold
text-gray-700
leading-relaxed
```

🎨 Colors

```
bg-blue-500
text-white
border-gray-300
```

✨ Effects

```
shadow-md
rounded-xl
hover:scale-105
transition
```

📱 Responsive

```
text-base
md:text-lg
lg:text-xl
```

🌙 Dark mode

```
dark:bg-gray-900
dark:text-white
```

💡 Tip

Compose tiny utilities into rich UIs without custom CSS

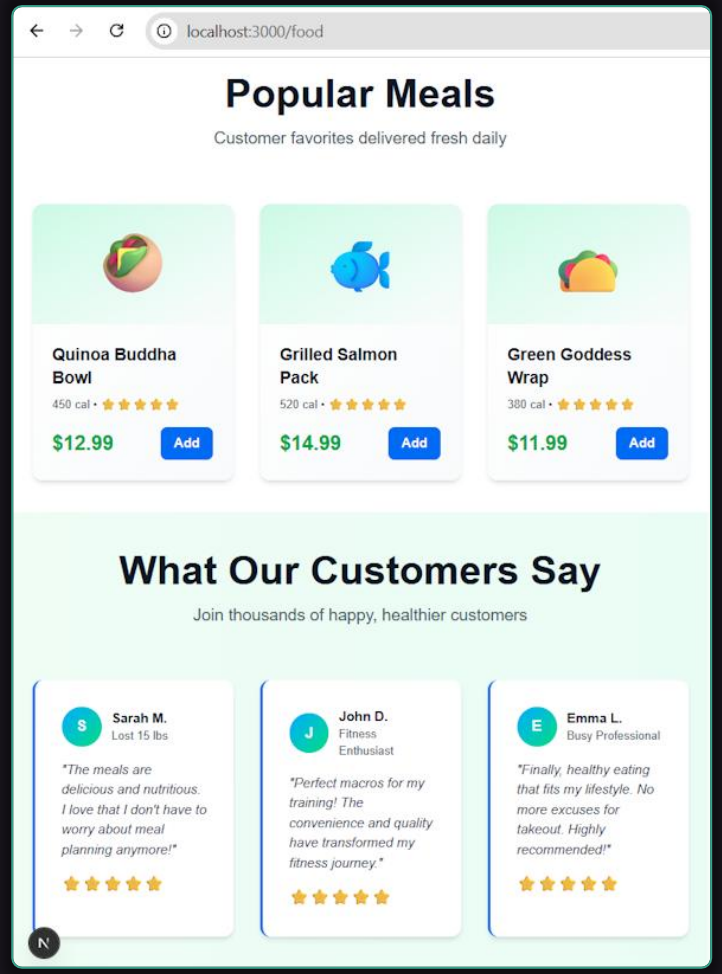
Live Demo – Landing Page with Tailwind

SoftUni AI

🤖 **AI prompt** to build a landing page with Tailwind:

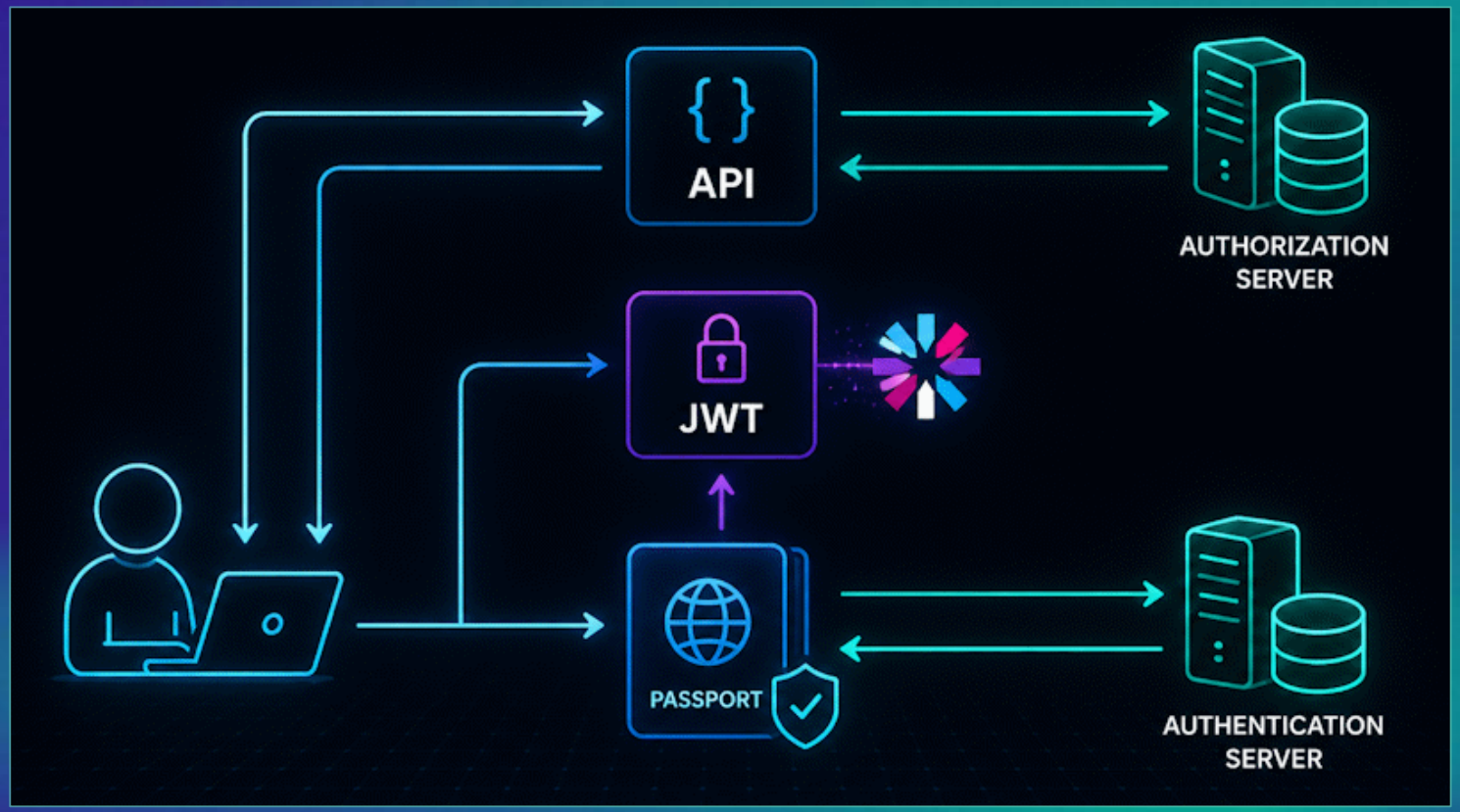
Create a modern, responsive **landing page** about "Healthy Food" business at `app/food/page.tsx`:

- Promote a healthy food and nutritious meals
- **Hero section**: large heading, subheading, CTA
- Sections: "**Features**" (3-column responsive grid), "**How It Works**", "**Popular Meals**", "**Testimonials**", "**Benefits**", "**Final CTA**"
- Implement **modern UI**, responsive design, with transitions, effects, icons / emojis, etc.
- Use only **Tailwind classes**, no custom CSS



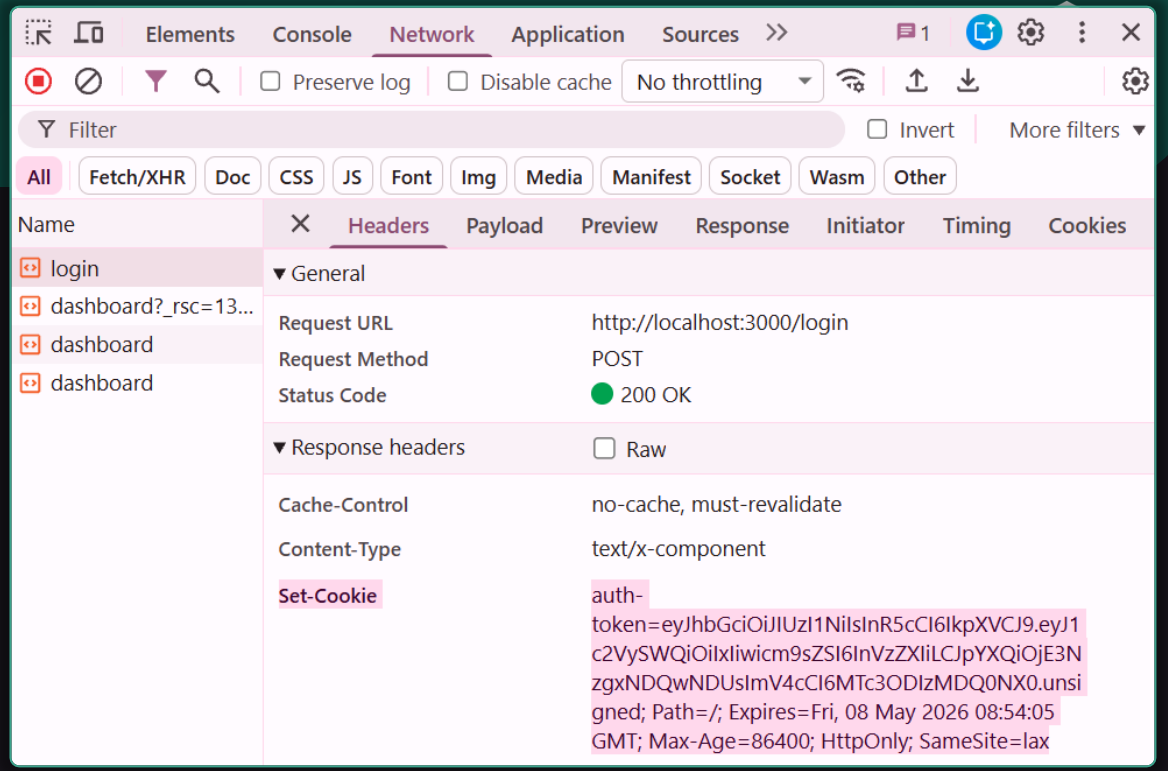
Auth, JWT and Middleware

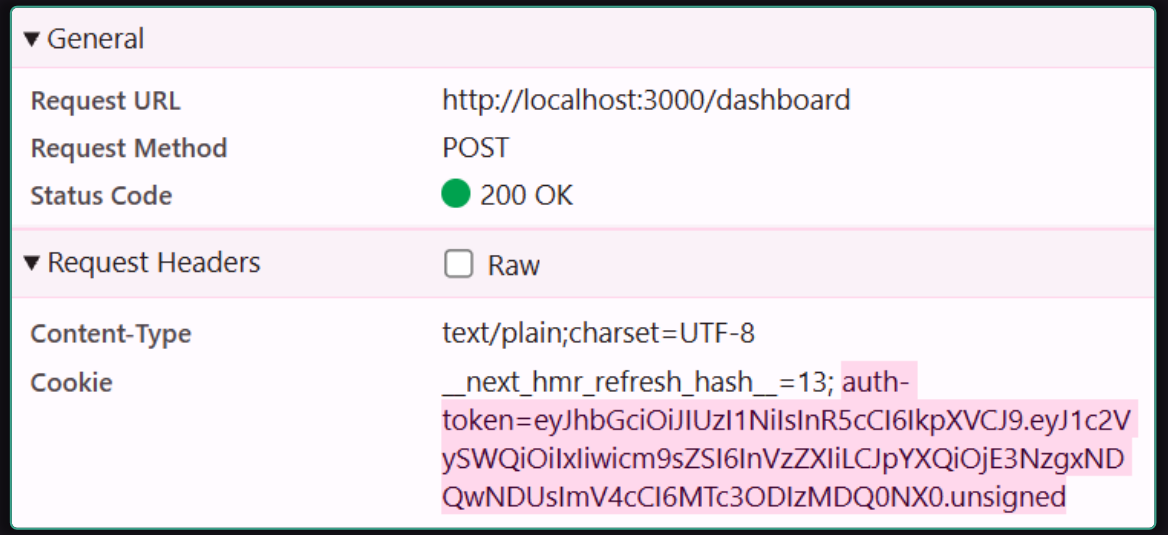
Cookies, Sessions, JWT, Role-Based Access



HTTP Cookies

- What are **HTTP cookies**?
  - Small **key-value data** strings
  - Stored in the **browser**
  - Sent by the **server** via a `**Set-Cookie**` HTTP response header
- **Request flow** with cookies
  - Browser **automatically attaches matching cookies**
  - To every subsequent HTTP request via the `**Cookie**` header





HTTP Cookies: Uses and Attributes

SoftUni AI

- **Common uses** of cookies
  - Sessions, preferences, carts, tracking
- Cookie **key attributes**
  - **Expires** / **Max-Age**, **HttpOnly**, **Secure**, **SameSite**
- Cookie **scope**
  - **Domain** + **Path**
  - Restrict which origins receive them
- Learn more at: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Cookies

Auth Concepts: Headers & Cookies

SoftUni AI

- In software systems **auth** is typically done with **JWT tokens**

⚙️ RESTful APIs: **HTTP header** brings the auth token

HTTP Response (Login)
`POST /api/login { user, pass }`
`200 OK { "token": "…" }`

HTTP Request (with Auth)
`Authorization: Bearer <token>`
`GET /api/protected-resource`

🌐 Browser-based apps: **HTTP cookie** brings the auth token

HTTP Response (Login)
`POST /login { user, pass }`
`Set-Cookie: session=<token> …`

HTTP Request (with Auth)
`Cookie: session=<token> …`
`GET /dashboard/protected-page`

JWT Authentication Flow for Web

SoftUni AI

1. Login
Email + password

2. Sign JWT
{ id + role } baked in

3. Set Cookie
httpOnly cookie

4. Auto-Send
Cookie on every request

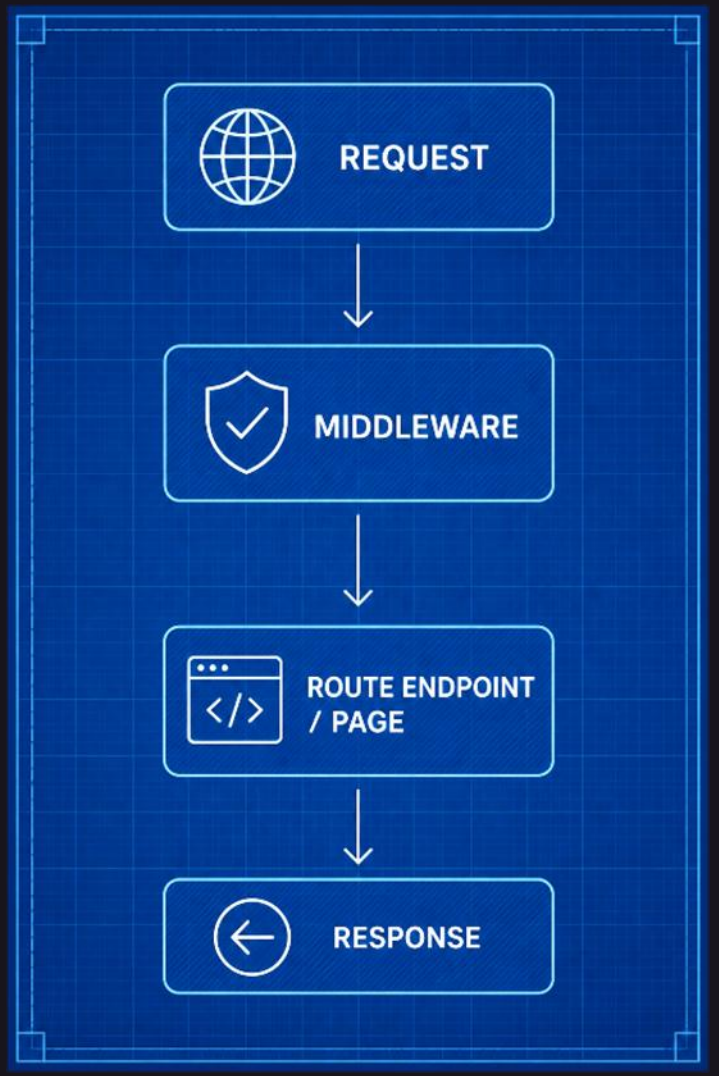
5. Decode JWT
Middleware reads it

6. Allow / Block
Based on role

7. Render UI
User-specific UI

Next.js Middleware

- In Next.js `middleware.ts` / `proxy.ts` runs **before every request** reaches a route
  - Placed at the **project root**
- Middleware **use cases**:
  - 🔐 **Authentication** (check JWT, redirect to login)
  - 👮 **Authorization** (role-based access)
  - 🧪 **A/B testing** (rewrite URLs)
  - 📊 **Logging**, analytics, geo-location
  - 🌍 **Locale detection** and redirects



Middleware – Example

SoftUni AI

- Auth middleware
  - Reads `token` from **cookies**
  - Redirects unauthenticated users to **/login**
  - `matcher` limits which paths to process

```
// middleware.ts
import { NextResponse }
from "next/server";
import type { NextRequest
} from "next/server";
```

```
export function middleware(req: NextRequest) {
  let token = req.cookies.get("token")?.value;
  if (req.nextUrl.pathname.startsWith(
      "/dashboard")) {
    if (!token) {
      return NextResponse.redirect(
        new URL("/login", req.url));
    }
  }
  return NextResponse.next();
}

export const config = {
  matcher: ["/dashboard/:path*",
    "/admin/:path*"],
};
```

Reading User in Server Components

SoftUni AI

- Server components can read `cookies()` directly
- Use `verifyJWT()` to decode the token
- **Note**: using `cookies()` makes the route **dynamic (SSR)**

```
import { cookies } from "next/headers";
import { verifyJWT } from "@/lib/auth";
```

```
export default async function Dashboard() {
  const token = (await cookies())
    .get("token")?.value;
  const user = token
    ? verifyJWT(token)
    : null;

  return (
    <h1>Hello, {user?.name ?? "Guest"}!</h1>
  );
}
```

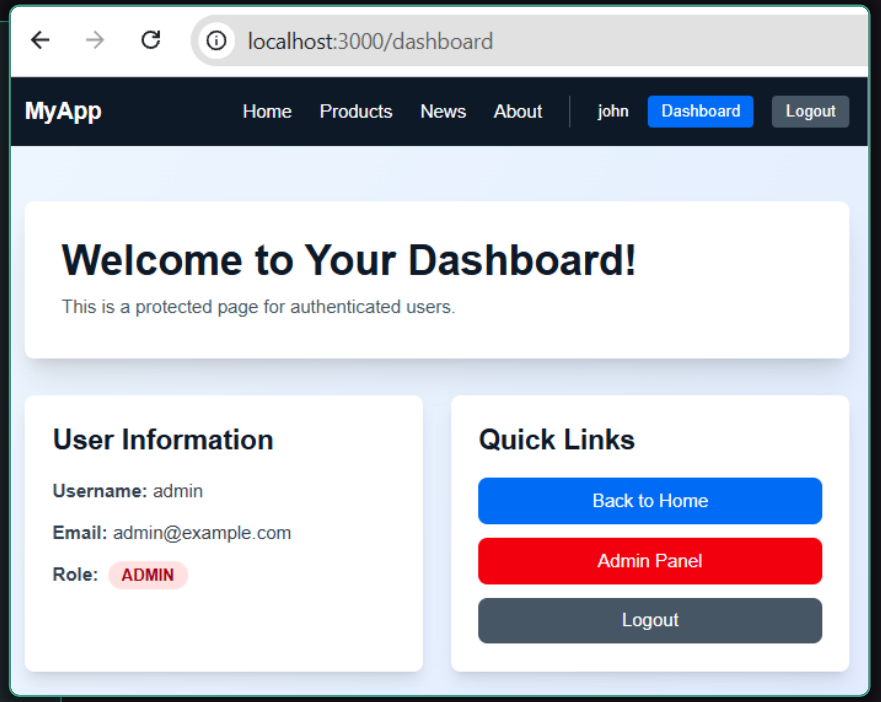
Live Demo – Role-Based App

SoftUni AI

🤖 **AI prompt** to implement users + roles with middleware:

In the **Next.js app** implement 3 areas:
- `/` → public home page
- `/dashboard` → protected, **logged-in** users
- `/admin` → protected, **admins** only

Auth setup:
- Use **server actions** for auth pages
- Keep users + roles in a local `users.json`
- `/login` sets a **JWT cookie** on success
- `middleware.ts` parses JWT → {`userId, role`}
- Redirects unauthenticated → `/login`
- Blocks non-admins from `/admin (403)`



Security Warning: JWT_SECRET

SoftUni AI

- Critical **security warning**:
  - Who knows your **JWT_SECRET** can **login without a password**!
  - Initialize properly **JWT_SECRET** in **`.env`**
  - Make the app **fail** at runtime if **JWT_SECRET is missing**

TS auth.ts U

lib > TS auth.ts > str2ab

```
1   // Bad code --> JWT_SECRET should not be hardcoded
2   const JWT_SECRET = process.env.JWT_SECRET || 'your-secret-key-change-in-production';
```

Fix security: the app should fail to start if JWT_SECRET is missing in `.env`

Claude Haiku 4.5

Performance Optimizations

Link Prefetch, Image Optimization, Caching

Next.js is Built for Performance

SoftUni AI

- Next.js is built for **performance** and **SEO**
  - **Server-side rendering** (SSR) → **loads faster** than client-side React
  - **Static pages** (SSG) and **cached pages** (ISR) → **served from CDN**
  - **Pre-fetched links** for faster navigation
  - **Image optimization** (smaller file size)
  - **Data caching** → reduce API / DB calls
  - **Suspense** and **streaming** → display the page in chunks (ready parts immediately, slower parts when loaded) → smoother UX

Faster Navigation with <Link>

SoftUni AI

- Use `next/link` instead of `<a>` for internal navigation
- **Benefits** of **<Link>**
  - **Client-side page transitions** (no full reload)
  - **Automatic prefetching** when link enters the viewport
  - **Smooth**, **SPA-like UX**
- Only use `<a>` for **external links**

```
import Link from "next/link";

<Link href="/about">About Us</Link>

<Link
  href="/dashboard"
  prefetch={true}
>
  Dashboard
</Link>
```

Live Demo – Links in Next.js

SoftUni AI

- At the app Home page:
  - Insert **<Link href=…>**

```
<Link href="/about">About Us</Link>
```

  - Insert **<a href=…>**

```
<a href="/about">Slow Link: About</a>
```

  - Insert **<Link href=… prefetch={true}>**

```
<Link href="/contacts"
prefetch={true}>Contacts</Link>
```

- Note: **prefetch** works on **production** only

```
npm run build
npm run start
```

Image Optimization with <Image>

SoftUni AI

- `next/image` automatically optimizes images:
  - 🗜️ Converts to modern formats (**WebP, AVIF**)
  - 🧮 **Resizes** to device size (responsive)
  - 💤 **Lazy-loads** off-screen images
  - 📐 Prevents **layout shift** (requires width + height)

```tsx
import Image from "next/image";

<Image
  src="/hero.jpg"
  alt="Hero banner"
  width={1200}
  height={600}
  priority   // load immediately
/>
```

Caching in Next.js

SoftUni AI

- **Three cache layers**:
  - **Full Route Cache** – static HTML of SSG/ISR pages
  - **Data Cache** – results of **fetch()** calls (opt-in/opt-out)
  - **Request Memoization** – dedupe identical fetches
- **Revalidate on demand**:
  - `revalidatePath("/books")`
  - `revalidateTag("books")`

```
// Cache control on fetch()
fetch(url)
  // cached forever (default)
```

```
fetch(url, { cache: "no-store" })
  // never cached → SSR
```

```
fetch(url,
  { next: { revalidate: 60 } })
  // ISR, 60 seconds
```

```
fetch(url,
  { next: { tags: ["books"] } })
  // tag-based revalidation
```

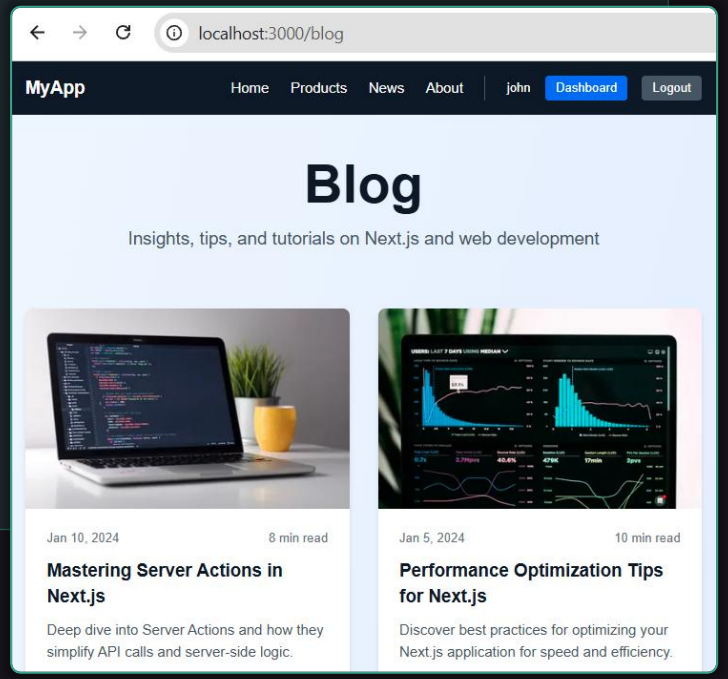
Live Demo – Optimized Blog Page

SoftUni AI

🤖 **AI prompt** to build a performance-optimized page:

Build a performance-optimized **blog index page**: **/blog** (SSR)

- Grid of **12 article cards**: image + title + excerpt
- `<Image>` for thumbnails (WebP, lazy loading)
- `<Link>` for each card -> **/blog/[slug]**
- **Prefetch** visible cards
- For simplicity, use `**blog-data.json**` (no database)

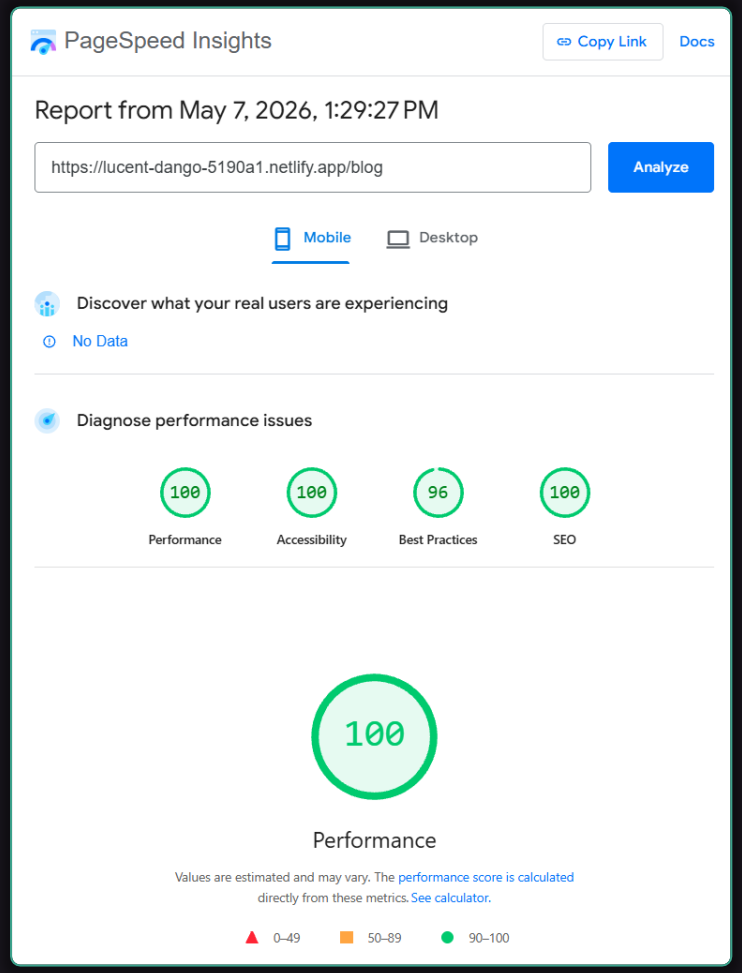


- **Deploy** the blog to Netlify
- Measure **page speed**: https://pagespeed.web.dev

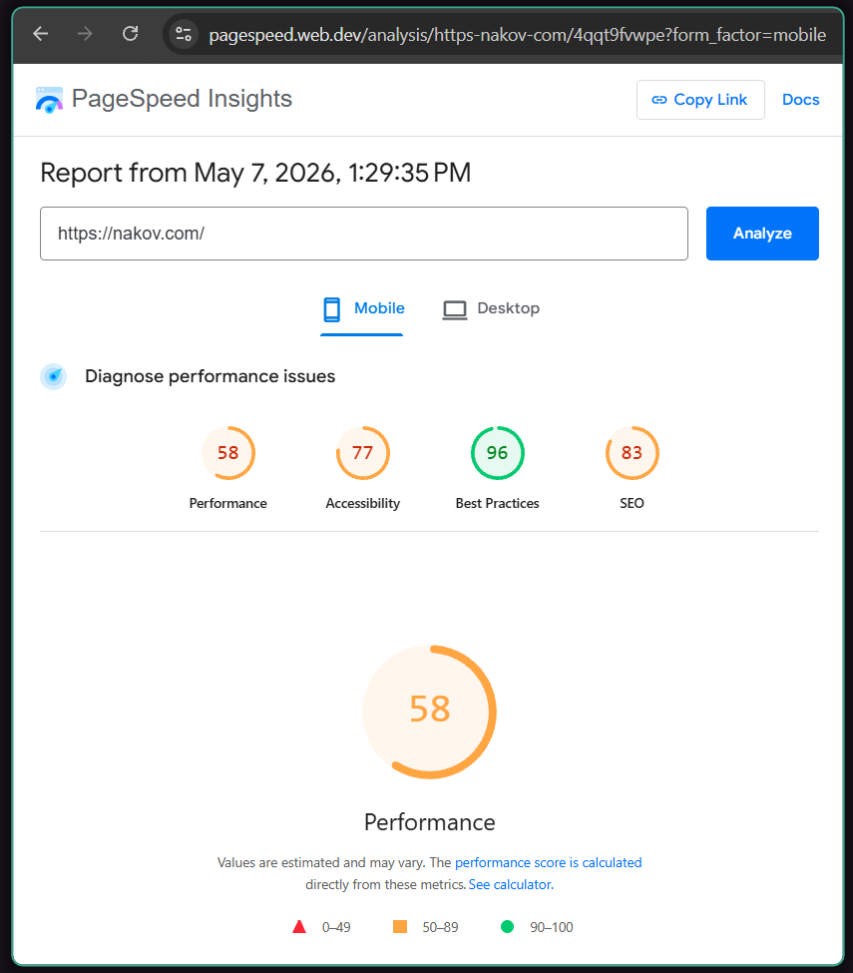
Optimized Blog in Page Speed

SoftUni AI

- Next.js site @ Netlify

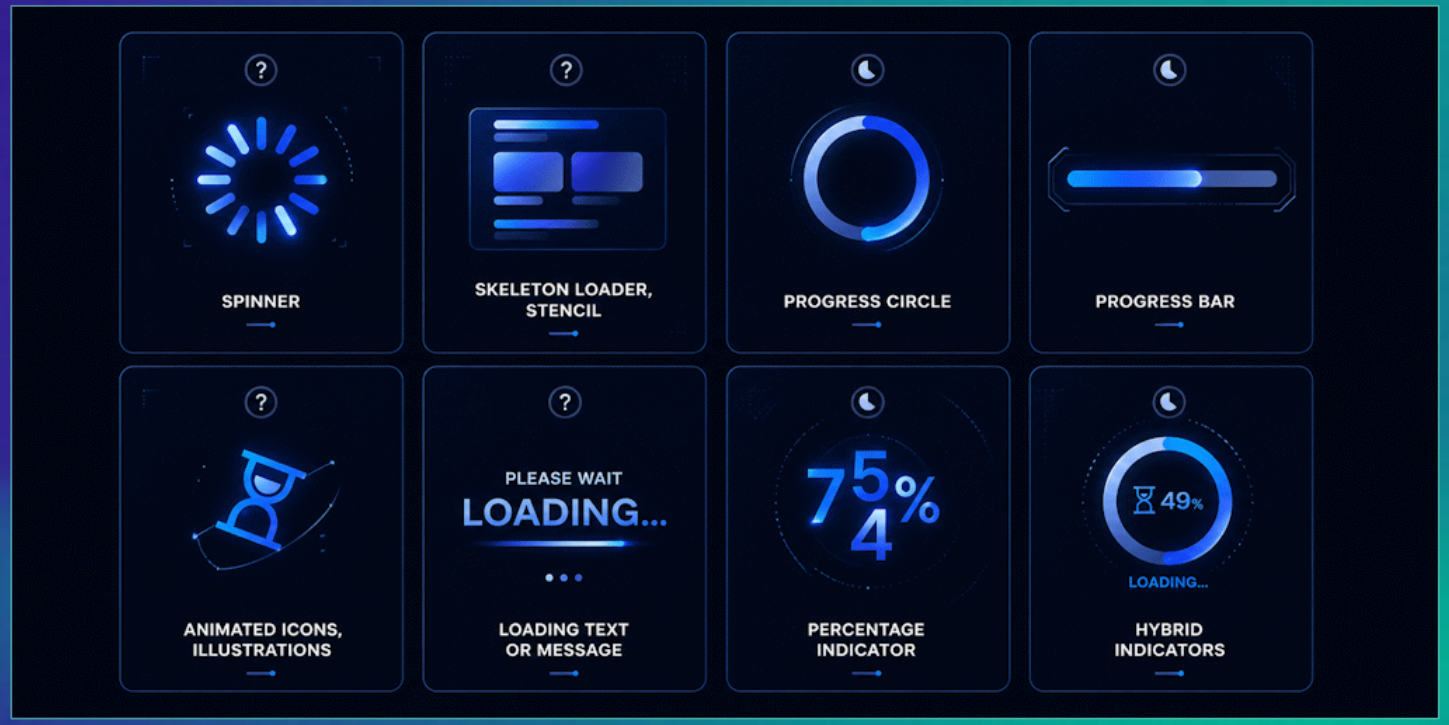


- WordPress site @ VPS



Suspense and Streaming

Progressive HTML, Loading States, Better UX



Why Suspense and Streaming?

SoftUni AI

⚠️ Problem

One **slow component** blocks the **whole page** from rendering

💡 Solution

Split the page into **chunks** and **stream** them as they become ready

⏸️ Suspense

Decides what can **wait** and what to **show** in the meantime

📡 Streaming

Decides when HTML **chunks are sent** to the browser

🚀 Result

User sees **fast content immediately**

Slow parts fill in **progressively**

<Suspense> in Next.js

SoftUni AI

- **Wrap slow components with <Suspense>** to stream loading
- Example:
  - Header shows **instantly**
  - Chart **streams in** when ready
- Each **<Suspense>** = independent stream

```
import { Suspense } from "react";
import SlowChart from "./SlowChart";
import FastHeader from "./FastHeader";
```

```
export default function DashboardPage() {
  return (
    <>
      <FastHeader />
      <Suspense
fallback={<p>Loading…</p>}>
        <SlowChart />
      </Suspense>
    </>
  );
}
```

Loading UI with `loading.tsx`

SoftUni AI

- A special file `loading.tsx` auto-wraps the route in `<Suspense>`
- **Shown instantly** while `page.tsx` fetches data on the server
- **Zero JS** needed – works with **Server Components**

```tsx
// app/dashboard/loading.tsx

export default function Loading() {
  return (
    <div className="p-8 animate-pulse">
      <p>Loading dashboard…</p>
    </div>
  );
}
```

- **Route-level** loading **progress indicator** (for all route pages)

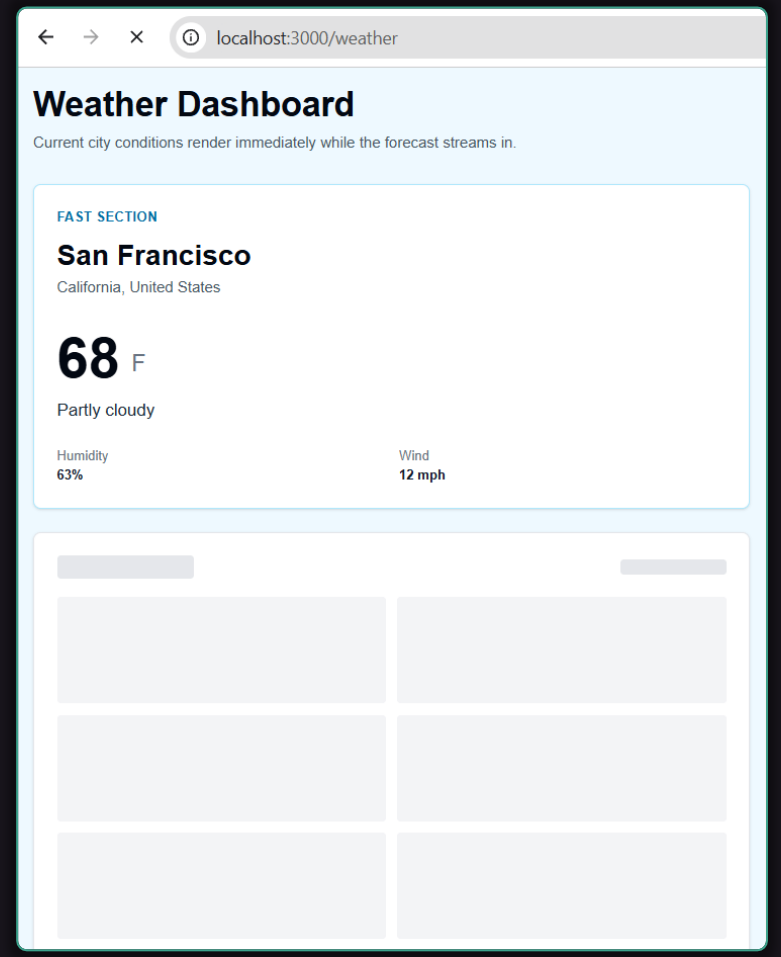
Live Demo – Weather Dashboard

SoftUni AI

🤖 AI prompt to build a dashboard with <Suspense>:

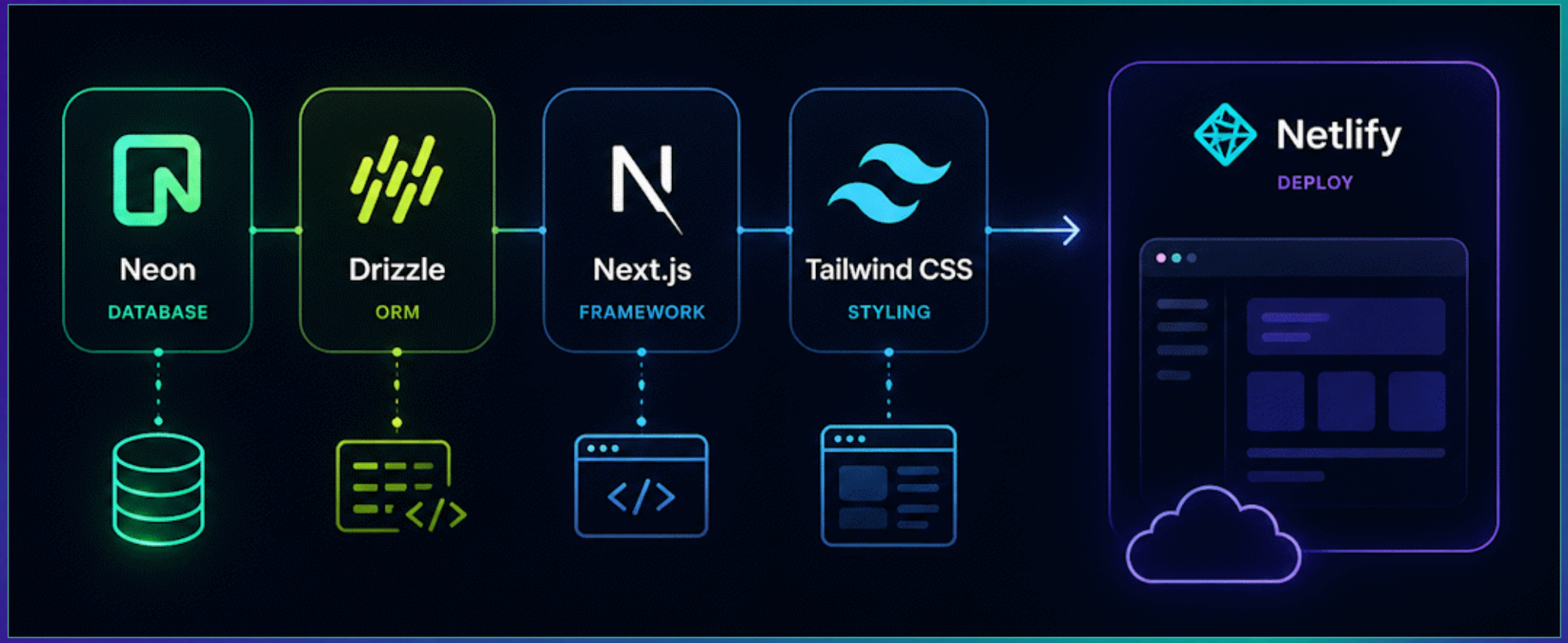
Build a **weather dashboard** at `/weather`:

- **Fast section**: city (renders instantly)
- **Slow section**: 7-day forecast (intentionally wait 3s at server-side)
- Wrap slow parts in `<Suspense>` with skeleton fallbacks
- Add `loading.tsx` as fallback
- For simplicity, use `**weather-data.json**` (no database)



Full-Stack App with Next.js

Next.js + Tailwind + Drizzle + Neon → Netlify



App Overview – "BlogHub" 

SoftUni AI

- What shall we build?
- **Full-stack blog app**:
  - 🛠️ **Back-end**: TypeScript + Next.js (server components) + Drizzle ORM + Neon PostgreSQL
  - 🎨 **Front-end**: TypeScript + Next.js + React + Tailwind
- **Features**:
  - 📋 List all blog posts (**dynamic route, ISR**)
  - 📖 View a single post (**dynamic route, ISR**)
  - ✍️ Register, Login (**server action + form**)
  - 📝 Create a new post, 🗑️ Delete a post (**server action + form**)

Step 1: Create Next.js Project

SoftUni AI

- Create a project in VS Code: `**nextjs-blog**`
- Scaffold an empty **Next.js project**:

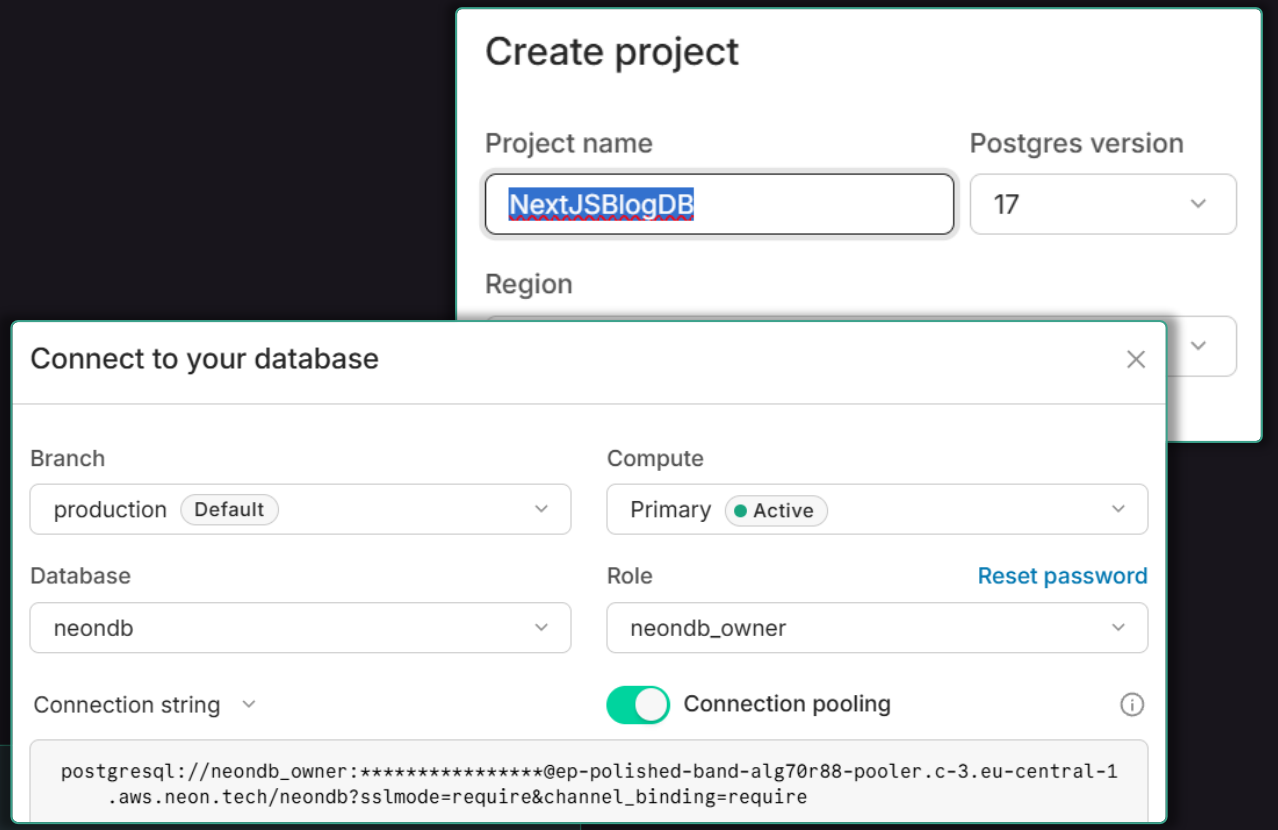
```
npx create-next-app@latest .
```

```
PS nextjs-blog-example> npx create-next-app@latest .
>>
√ Would you like to use the recommended Next.js defaults? » Yes, use recommended defaults
Creating a new Next.js app in C:\Projects\Full-Stack-Apps\backend-apis\nextjs-blog-example.

Using npm.

Initializing project with template: app-tw
```

Step 2 – Create a Neon Database

SoftUni AI

- Go to https://neon.tech
  → sign up (free tier)
- Create a project:
  NextJSBlogDB
- Copy the connection string → paste into `.env`



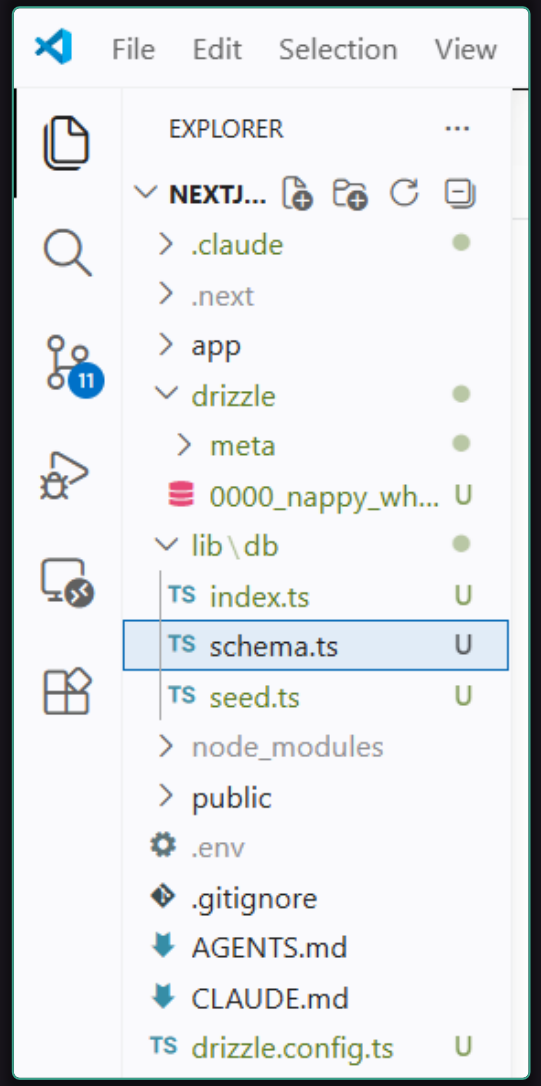

```
# .env
DATABASE_URL="postgresql://user:pass
  @ep-xxx.neon.tech/NextJSBlogDB"
```

Step 3 – Drizzle ORM Setup

SoftUni AI

🤖 **AI prompt** to setup Db with Drizzle:

Set up **Drizzle ORM** in this project:
- Install **Drizzle** ORM + **Neon** driver from NPM
- Create **Drizzle schema**:
  - **users** (email, passwordHash)
  - **posts** (title, content, tags[], date, owner → user)
- Create `**db:generate**` and `**db:migrate**` npm scripts (use Drizzle Kit)
- **Generate** an **migrate** the schema to DB



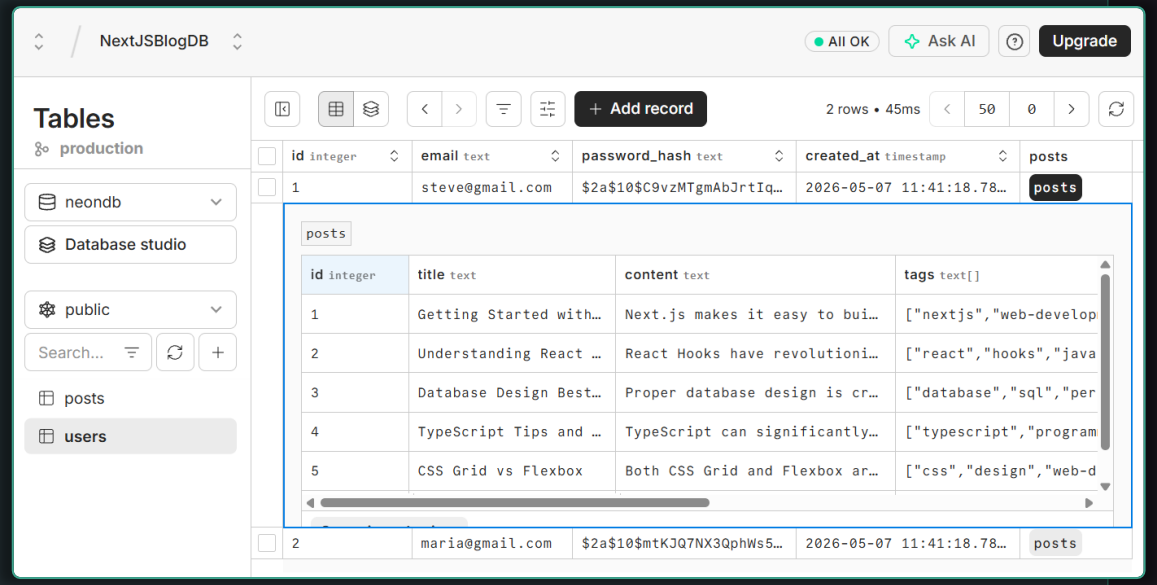
Step 4 – Seed Sample Data in DB

SoftUni AI

🤖 **AI prompt** to seed sample data in the DB:

Seed sample data in the DB:

- Define npm script `**db:seed**`
- Create **users**:
  { steve@gmail.com, pass123 },
  { maria@gmail.com, pass123 }
- Hash the passwords with **bcrypt**
- Insert 5-6 **blog posts** per user
- **Run** the DB seed script to seed DB



Step 5 – Build App Pages + Actions

SoftUni AI

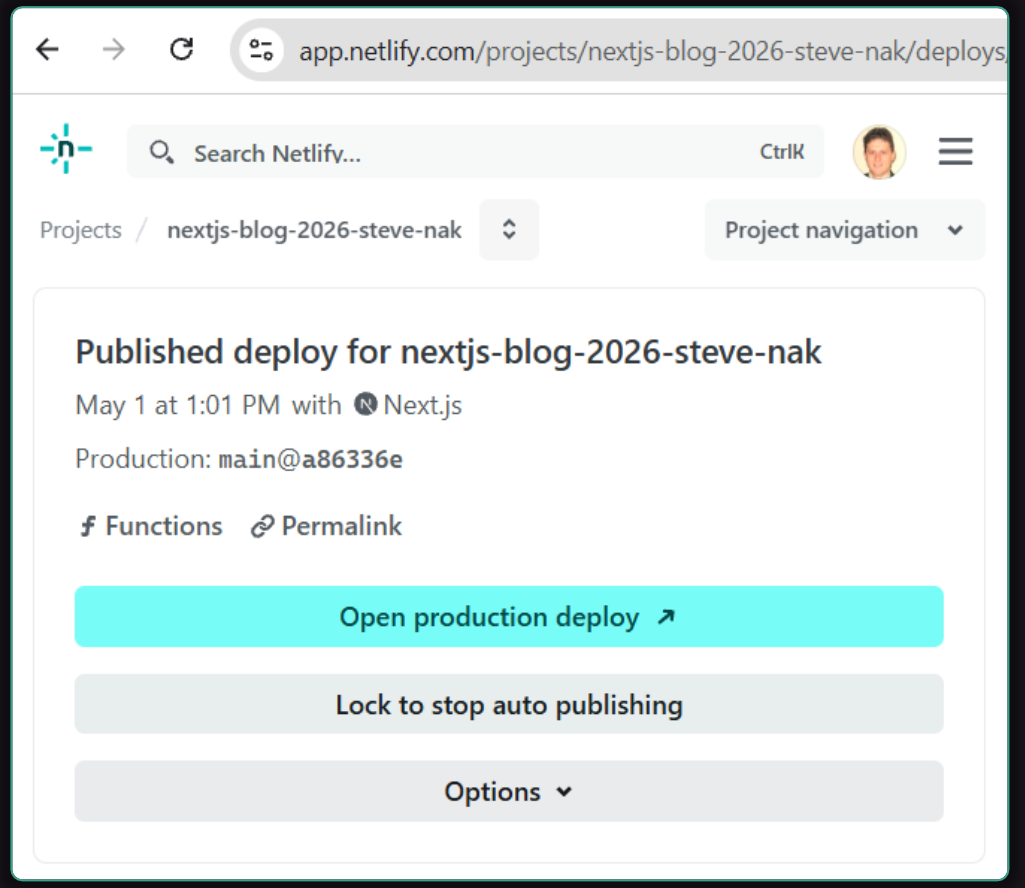
🤖 **AI prompt** to build the app functionality:

Build these **pages and routes**, using Next.js App Router (layout + pages + server actions):
App pages:
/ -> list all posts (ISR, 60s)
/posts/[id] -> single post (ISR, 60s)
/register, **/login**, **/logout** -> auth (SSG + client form + server action)
/posts/new -> form to create a post
Server actions:
- **login** / **register** / **logout** -> JWT token, bcrypt
- **createPost(formData)** -> insert, revalidate /, /posts
- **deletePost(id)** -> delete, revalidate /, /posts
Style with **Tailwind**: modern, responsive design, with **suspense** + **loading.tsx**

Step 5 – Deploy to Netlify 🚀

SoftUni AI

- Push the repo to **GitHub**
- Go to **https://netlify.com** → "Add new site" → "Import from Git"
- Netlify **auto-detects Next.js**
- **Add environment variables**:
  - `DATABASE_URL` = Neon connection string
  - `JWT_SECRET` = random secret
- Click **"Deploy"** → live at `<name>.netlify.app`
- Every `git push` triggers a re-deploy



Lesson Summary

SoftUni AI

- **Next.js** = full-stack React framework with **SSR**, **SSG**, **ISR**
- **App Router** uses file-system routing: folders = routes
- **Server Components** by default — DB access, SEO-friendly
- **Client Components** (`"use client"`) — interactivity, hooks, events
- **Server Actions** replace most `/api` routes for mutations
- **Middleware** runs before every request — perfect for **auth**
- `<Link>` and `<Image>` give free performance wins
- **Suspense + streaming** deliver fast UX even with slow data
- Deploy to **Netlify / Vercel** with zero config

Questions?

Postbank – Exclusive Partner for SoftUni AI

- One of the leading **banking institutions** in Bulgaria
- Member of the Eurobank Group with € 103 billion assets
- Innovative trendsetter with next generation **beyond banking**, transforming today, empowering tomorrow
- Certified **Top Employer 2025** by the international Top Employers Institute
- Proven people care and **wellbeing initiatives**
- Benefits and unlimited access to professional, **personal and leadership trainings and programs**
- www.postbank.bg / careers.postbank.bg

Diamond Partners of Software University

Software University

Coca-Cola HBC Bulgaria

Postbank

top EMPLOYER България Bulgaria 2025

FOR A BETTER WORLD OF WORK

SUPER HOSTING .BG

encorp.ai

createX

DRAFT KINGS

THE CROWN IS YOURS

INDEAVR

Serving the high achievers

Diamond Partners of SoftUni Digital

SoftUni Digital

Coca-Cola HBC Bulgaria

Postbank

top EMPLOYER България Bulgaria 2025

FOR A BETTER WORLD OF WORK

SUPER HOSTING .BG

encorp.ai

createX

INDEAVR

Serving the high achievers

DRAFT KINGS

THE CROWN IS YOURS

Diamond Partners of SoftUni Digital

SoftUni Digital

HUMAN

NETPEAK
DIGITAL GROWTH PARTNER

MarmaLAB | E-комерс агенция

ETIENYANEV
Break Your *Limits*. Live Your *Brand*.

1 FOR FIT

Aleikov Studio

Diamond Partners of SoftUni Creative

SoftUni Creative

Coca-Cola HBC Bulgaria

Postbank

top EMPLOYER България Bulgaria 2025

FOR A BETTER WORLD OF WORK

SUPER HOSTING .BG

encorp.ai

createX

DRAFT KINGS

THE CROWN IS YOURS

INDEAVR

Serving the high achievers

Organization Partners of SoftUni Creative

SoftUni Creative

creative corner

Studio π

Think Big • Design Smart

THE BUCKS TOWN'S WORK

DEFFECTO